11. Difference Between Agent and Node in Jenkins Pipeline

Think of Jenkins as a **factory manager** that needs to get work done (building software). To do this work, it needs:

- **Workers** (machines/computers) — called **Nodes**
- **Instructions** on where and how to do the work — called **Agents**

1. What is a Jenkins Pipeline?

A **pipeline** is just a **step-by-step recipe** for building, testing, and deploying software.

Example recipe:

1. Get the code
2. Build it
3. Test it
4. Deploy it

Each step is called a **stage**. Inside each stage, you have **steps** (actual commands).

A pipeline has these parts:

- **stages** — the big phases (Build, Test, Deploy)
- **steps** — the actual commands inside stages
- **agent** — where the work happens
- **node** — the actual machine doing the work

2. What is an Agent?

Agent = "Where should this pipeline (or stage) run?"

It's like telling Jenkins: "Run this job on any available machine" or "Run this on a Linux machine" or "Run this inside a Docker container."

Types of Agents:

Type	Meaning
any	Run on any available machine
label	Run on a machine with a specific tag (e.g., "linux", "windows")
docker	Run inside a Docker container
none	No global agent — you'll specify per stage

3. What is a Node?

Node = The actual physical or virtual machine that does the work.

Jenkins uses a **master-slave architecture**:

- **Master** = boss (schedules and manages tasks)
- **Node (slave)** = worker (actually runs the tasks)

Node Characteristics:

- **Labels** — tags like "linux", "windows", "docker" to identify nodes
- **Workspace** — the folder on the node where files are stored
- **Executor** — how many tasks a node can run at the same time

4. Key Differences (Simple Table)

Feature	Agent	Node
What it is	Where work runs (abstract)	Actual machine (concrete)
Scope	Can cover multiple nodes	One specific machine
Usage	In pipeline/stage config	Inside script { node(...) } blocks
Flexibility	High-level control	Granular control
Analogy	"Send the package to any warehouse"	"Send the package to Warehouse #5"

5. Real-World Use Cases

Use Case 1: Agent for Different Stages

You might want:

- **Build stage** → run on any machine (agent any)
- **Test stage** → run on a special machine with testing tools (agent { label 'test-node' })

Use Case 2: Agent with Docker

For isolated, consistent environments:

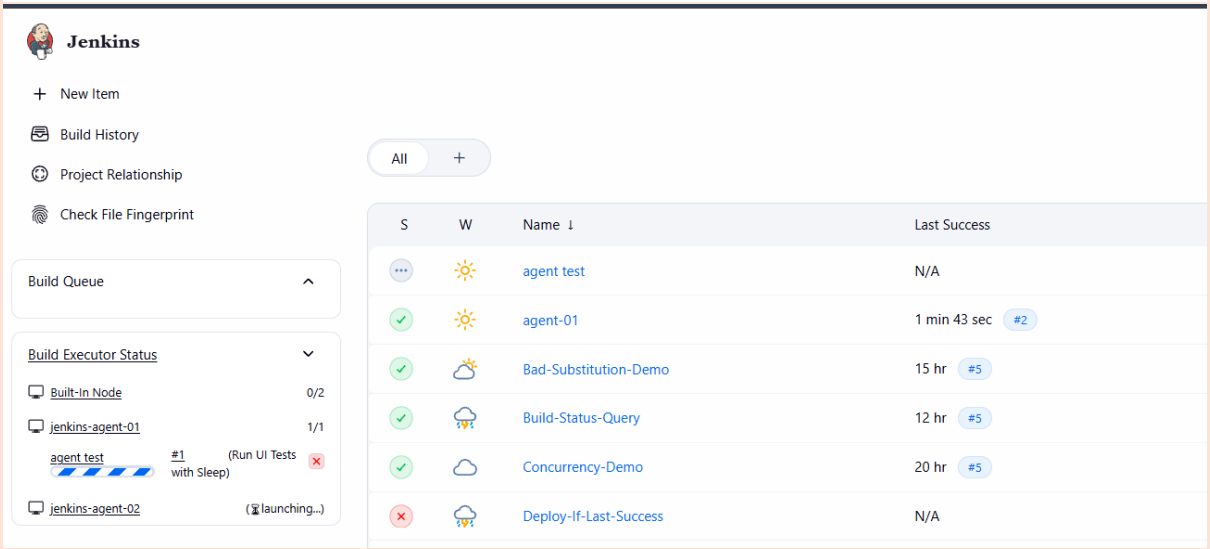
```
agent {
    docker { image 'maven:3-alpine' }
}
```

Use Case 3: Node for Granular Control
When you need a specific machine for a task:

```
script {
    node('build-node') {
        echo 'Building on build-node...'
    }
}
```

Implementation:





6. Summary — The One-Liner

Agent = the *instruction* about where to run (abstract).
Node = the *actual machine* that runs the work (concrete).

You use **agents** in your pipeline to say *where* things should run.
You use **nodes** (often via agents) to actually *execute* the work on real machines.

Memory Trick

- **Agent** → "**A**sking where to go" (instruction)
- **Node** → "**N**ow doing the work" (machine)

12. Reading a File From Workspace in Jenkins Using Groovy (Easy Explanation)

1. What is a Jenkins Workspace? 📁

Workspace = A dedicated folder on the Jenkins machine where the build happens.

When Jenkins starts a build, it automatically creates or updates this folder. Think of it as the **workbench** where everything happens.

What's inside a workspace?

- **Source code** — pulled from Git/SVN
- **Build artifacts** — compiled files, test reports, logs

Why are workspaces important?

- **Isolation** — each job gets its own workspace, so jobs don't clash
- **Incremental builds** — Jenkins can reuse the same workspace, so it only updates changed files instead of re-downloading everything (saves time!)

🧠 **Analogy:** Each Jenkins job has its own desk. Nobody else's mess affects your work.

2. Groovy Basics for Jenkins 🐘

Groovy = A powerful scripting language that works seamlessly with Jenkins.

It's like Java but **simpler and shorter**. You can write entire pipelines in Groovy (in a Jenkinsfile) or use it for small tasks inside a pipeline step.

Simple Groovy example in a Jenkinsfile:

```
pipeline {
    agent any
    stages {
        stage('Example') {
            steps {
                script {
                    def message = "Hello from Groovy!"
                    echo message
                }
            }
        }
    }
}
```

This just prints: Hello from Groovy!

Key Groovy features for file operations:

Feature	What it does
new File('file.txt')	Creates a file object
.text	Reads the entire file as a string
.eachLine { }	Reads the file line by line (good for big files)
Closures { }	Blocks of code passed around (used in eachLine)
.withReader { }	Opens a file, auto-closes it when done

Examples:

```
// Read entire file
def content = new File('example.txt').text
println content

// Read line by line
new File('example.txt').eachLine { line ->
   println "Line: $line"
}
```

Extra Tip:

Yes, exactly. By default, the Jenkins Script Security Sandbox blocks many built-in Groovy helper methods (such as decodeBase64, encodeBase64) and raw Java classes (such as new File) because they are not whitelisted out of the box.

What is the Script Security Sandbox?

→The sandbox is a built-in Jenkins security feature designed to prevent arbitrary or malicious pipeline scripts from executing dangerous operations (like reading unauthorized files on the server, running unauthorized system commands directly from controller memory, or accessing restricted Java APIs).

→When a script tries to use a method that isn't on Jenkins' default safety whitelist, the sandbox intercepts it and throws a RejectedAccessException.

How to Handle This

You have two main ways to deal with restricted methods like Base64 decoding:

1. The Recommended Way: Use Shell Commands (sh)

As shown previously, running your logic inside a shell block (sh '''...''') completely bypasses the Groovy sandbox because the script is executed natively by the operating system shell on the agent, not interpreted by Jenkins' restricted Groovy engine.

2. The Admin Way: Whitelist the Signature

If you are a Jenkins administrator and **want** to use Groovy methods like decodeBase64 directly in scripts, you can manually approve them:

1. Go to **Manage Jenkins** > **In-process Script Approval**.
2. Find the pending signature request (e.g., staticMethod org.codehaus.groovy.runtime.EncodingGroovyMethods decodeBase64 java.lang.String).

3. Click **Approve**.

3. Understanding File Paths in Jenkins

By default, Jenkins runs build steps with the workspace as the current directory.

That means you can use **relative paths** (no need for full paths)

Examples:

```
// File in root of workspace
def config = readFile 'config.json'

// File in a subfolder
def logs = readFile 'logs/build.log'
```

Tip: Jenkins has **built-in steps** for file operations (like readFile) — these are preferred over raw Groovy file I/O because they work consistently across different Jenkins setups.

4. Ways to Read Files From Workspace

4.1. Using readFile (⭐ Recommended)

```
pipeline {
    agent any
    stages {
        stage('Create File') {
            steps {
                sh 'echo "1.0.0" > version.txt'
                sh 'echo "File created."'
            }
        }
        stage('Read with readFile') {
            steps {
                script {
                    def version = readFile('version.txt').trim()
                    echo "Current version: ${version}"
                }
            }
        }
    }
}
```

Simple, consistent, works everywhere in Jenkins.

- Reads the whole file into a variable

4.2. Using Groovy's File Class

```
pipeline {
    agent any
    stages {
        stage('Create File') {
```

```
        steps {
            sh 'echo "Hello from Jenkins workspace" > greeting.txt'
        }
    }
    stage('Read with readFile') {
        steps {
            script {
                // Use Jenkins built-in readFile step
                def content = readFile('greeting.txt')
                echo "File content: ${content.trim()}"
            }
        }
    }
  }
}
```

- More flexible for complex operations.
- Works in both Jenkins pipelines **and** standalone Groovy scripts.

4.3. Reading Large Files Line by Line

```
def file = new File('large_file.txt')
file.eachLine { line ->
  echo "Line: ${line}"
}
```

- **Memory-efficient** — doesn't load the whole file at once.
- Best for big files

```
pipeline {
  agent any
  stages {
    stage('Create Log File') {
        steps {
            sh '''
            echo "INFO: Starting build" > build.log
            echo "WARNING: Deprecated API used" >> build.log
            echo "ERROR: Compilation failed on line 42" >> build.log
            echo "INFO: Retrying..." >> build.log
            echo "WARNING: Slow query detected" >> build.log
            '''
        }
    }
    stage('Process Log Line by Line') {
        steps {
            script {
                def errorCount = 0
                def warningCount = 0

                // Read the file using Jenkins built-in readFile step
                def logContent = readFile('build.log')
```

```
                // Iterate line by line over the string safely
                logContent.eachLine { line ->
                    if (line.contains('ERROR')) {
                        errorCount++
                        echo "Error found: ${line}"
                    } else if (line.contains('WARNING')) {
                        warningCount++
                    }
                }

                echo "Analysis complete. Errors: ${errorCount}, Warnings: ${warningCount}"
            }
        }
    }
  }
}
```

4.4. **Method 4: Reading Specific Lines**

```
pipeline {
  agent any
  stages {
    stage('Create Multi-line File') {
      steps {
        sh '''
        echo "Line 1: Header" > data.txt
        echo "Line 2: Config" >> data.txt
        echo "Line 3: Value" >> data.txt
        echo "Line 4: Footer" >> data.txt
        '''
      }
    }
    stage('Read Specific Line') {
      steps {
        script {
          // Read file using Jenkins built-in step, then split/read lines safely
          def content = readFile('data.txt')
          def lines = content.readLines()

          echo "Total lines: ${lines.size()}"
          echo "First line: ${lines[0]}"
          echo "Third line: ${lines[2]}"
          echo "Last line: ${lines[-1]}"
        }
      }
    }
  }
}
```

Reads all lines into a list, then picks the one you want.

- ⚠ Remember: Groovy uses **0-based indexing** (first line = index 0).

4.5. **Method 5: Reading Binary Files**

```
pipeline {
    agent any
    stages {
        stage('Create Binary File') {
            steps {
                sh 'head -c 1024 /dev/urandom > binary.dat'
            }
        }
        stage('Read Binary File') {
            steps {
                sh '''
                    echo "===== Inspecting Binary File via Shell ====="

                    # Get file size in bytes
                    FILE_SIZE=$(wc -c < binary.dat)
                    echo "File size: ${FILE_SIZE} bytes"

                    # Get the first byte in decimal format
                    FIRST_BYTE=$(od -An -N1 -t u1 binary.dat | tr -d ' ')
                    echo "First byte: ${FIRST_BYTE}"
                '''
            }
        }
    }
}
```

- Use .bytes for binary files (images, zip files, etc.)

Method 6: Processing CSV Files

```
pipeline {
    agent any
    stages {
        stage('Create CSV') {
            steps {
                sh '''
                cat > deploy_config.csv <<EOF
env,url,credentials
dev,http://dev.example.com,dev-creds
staging,http://staging.example.com,staging-creds
prod,http://prod.example.com,prod-creds
EOF
                '''
            }
        }
        stage('Process CSV') {
```

```
        steps {
            script {
                // Read file safely using Jenkins built-in readFile step
                def configs = readFile('deploy_config.csv').readLines()

                // Skip header row and process each line
                configs.drop(1).each { line ->
                    if (line.trim()) {
                        def parts = line.split(',')
                        if (parts.size() >= 3) {
                            def env = parts[0].trim()
                            def url = parts[1].trim()
                            def credentials = parts[2].trim()
                            echo "Deploying to ${env} at ${url} using ${credentials}"
                        }
                    }
                }
            }
        }
    }
}
```

Method 7: Parsing JSON Files

```
pipeline {
    agent any
    stages {
        stage('Create JSON') {
            steps {
                sh '''
                cat > build_config.json <<EOF
{
    "projectName": "MyApp",
    "version": "2.5.1",
    "steps": [
        {"name": "compile"},
        {"name": "test"},
        {"name": "package"}
    ]
}
EOF
                '''
            }
        }
        stage('Parse JSON') {
            steps {
                script {
                    // Use Jenkins built-in readJSON step (sandbox-safe)
```

```
            def config = readJSON file: 'build_config.json'

            echo "Building ${config.projectName} version ${config.version}"
            config.steps.each { step ->
               echo "Executing step: ${step.name}"
            }
         }
      }
   }
}
```

7. Comparison Table of Reading Methods

Method	When to Use	Memory Efficient?	Plugin Required?
readFile 'file.txt'	Simple reads, consistent	✅ For small files	❌ No
new File('file.txt').text	Full content, flexible	Depends on file size	❌ No
.eachLine { }	Large files	✅ Yes	❌ No
.readLines()[n]	Specific lines	❌ Loads all lines	❌ No
.bytes	Binary files	❌ Loads all bytes	❌ No
readJSON (step)	Structured JSON	✅ Yes	✅ Pipeline Utility Steps

Implementation:

Method:1

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/read-file-demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Create File)
[Pipeline] sh
+ echo 1.0.0
[Pipeline] sh
+ echo 'File created.'
File created.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Read with readFile)
[Pipeline] script
[Pipeline] {
[Pipeline] readFile
[Pipeline] echo
Current version: 1.0.0
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Method:2

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/read-file-demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Create File)
[Pipeline] sh
+ echo 'Hello from Jenkins workspace'
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Read with readFile)
[Pipeline] script
[Pipeline] {
[Pipeline] readFile
[Pipeline] echo
File content: Hello from Jenkins workspace
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Method:3

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/read-file-demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Create Log File)
[Pipeline] sh
+ echo 'INFO: Starting build'
+ echo 'WARNING: Deprecated API used'
+ echo 'ERROR: Compilation failed on line 42'
+ echo 'INFO: Retrying...'
+ echo 'WARNING: Slow query detected'
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Process Log Line by Line)
[Pipeline] script
[Pipeline] {
[Pipeline] readFile
expected to call java.lang.String.eachLine but wound up catching org.jenkinsci.plugins.workflow.cps.CpsClosure2.call; see: https://jenkins.io/redirect/pipeline-cps-method-mismatches/
[Pipeline] echo
Analysis complete. Errors: 0, Warnings: 0
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Method:4

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/read-file-demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Create Multi-line File)
[Pipeline] sh
+ echo 'Line 1: Header'
+ echo 'Line 2: Config'
+ echo 'Line 3: Value'
+ echo 'Line 4: Footer'
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Read Specific Line)
[Pipeline] script
[Pipeline] {
[Pipeline] readFile
[Pipeline] echo
Total lines: 4
[Pipeline] echo
First line: Line 1: Header
[Pipeline] echo
Third line: Line 3: Value
[Pipeline] echo
Last line: Line 4: Footer
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Method:5

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/read-file-demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Create Binary File)
[Pipeline] sh
+ head -c 1024 /dev/urandom
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Read Binary File)
[Pipeline] sh
+ echo '===== Inspecting Binary File via Shell ====='
===== Inspecting Binary File via Shell =====
++ wc -c
+ FILE_SIZE=1024
+ echo 'File size: 1024 bytes'
File size: 1024 bytes
++ od -An -N1 -t u1 binary.dat
++ tr -d ' '
+ FIRST_BYTE=103
+ echo 'First byte: 103'
First byte: 103
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Method:6

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/read-file-demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Create CSV)
[Pipeline] sh
+ cat
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Process CSV)
[Pipeline] script
[Pipeline] {
[Pipeline] readFile
[Pipeline] echo
Deploying to dev at http://dev.example.com using dev-creds
[Pipeline] echo
Deploying to staging at http://staging.example.com using staging-creds
[Pipeline] echo
Deploying to prod at http://prod.example.com using prod-creds
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Method:7

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/read-file-demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Create JSON)
[Pipeline] sh
+ cat
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Parse JSON)
[Pipeline] script
[Pipeline] {
[Pipeline] readJSON
[Pipeline] echo
Building MyApp version 2.5.1
[Pipeline] echo
Executing step: compile
[Pipeline] echo
Executing step: test
[Pipeline] echo
Executing step: package
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Conclusion

→Reading Jenkins workspace files with Groovy is a core pipeline skill. You can use the simple **readFile** step or the flexible **Groovy File class**, while **eachLine** efficiently handles large files. Structured data like CSV and JSON can be parsed using native tools like **split** and **JsonSlurper**.

→These techniques let pipelines analyze logs, parse configs, and inspect artifacts. When combined with status controls, your pipeline can make decisions and flag issues without crashing. Best of all, these core capabilities require **no extra plugins**.

==========

Task 13: How to Create Dynamic Stages in a Jenkins Pipeline

1. Objective

To generate pipeline stages at runtime — instead of hardcoding them — using Groovy loops, conditionals, config files, and parallel execution, so that a single Jenkinsfile can adapt to different branches, environments, and configurations.

2. Purpose

Traditional Jenkins pipelines define a fixed set of stages that run identically on every build. This is simple but rigid. Real-world projects often need pipelines that adapt based on:

- **Git branch** — main deploys to production, feature/* deploys to dev, hotfix/* deploys to staging
- **Environment list** — deploy to dev, qa, staging, prod — all from one list
- **Configuration files** — external YAML that defines what steps to run
- **Test suites** — generate parallel test stages dynamically

- **External inputs** — respond to parameters or API payloads

Dynamic stages solve this by generating stage structure **at runtime**. Benefits include:

- **Flexibility** — stages change based on project state
- **Reusability** — same Jenkinsfile works across projects
- **Maintainability** — change logic in one place, affects all generated stages
- **Speed** — parallel dynamic stages run simultaneously
- **Externalization** — move pipeline logic to YAML files editable without touching the Jenkinsfile

3. Plugins Needed

Feature	Plugin Required
Pipeline (Declarative)	Pipeline plugin (usually pre-installed)
Stage View	Pipeline: Stage View plugin (usually pre-installed)
readYaml step	Pipeline Utility Steps plugin
parallel step	None — built into Pipeline
Groovy loops and closures	None — part of Groovy runtime
Git branch detection	Git + Multibranch Pipeline plugin

4. Concepts Reference

4.1 Static vs Dynamic Stages

Aspect	Static Stages	Dynamic Stages
Definition	Hardcoded in Jenkinsfile	Generated at runtime
Adaptability	Same every run	Changes based on conditions
Complexity	Simple	More advanced
Reusability	Limited	High
Best For	Simple, consistent workflows	Complex, adaptive workflows

4.2 Three Main Techniques

Technique	Use When
Groovy scripts (each, collectEntries)	Looping over lists, applying conditions
Jenkins DSL with parallel	Running independent tasks simultaneously
Config files (YAML)	Externalizing pipeline structure

4.3 Real-World Use Cases

Use Case	Benefit
Git branch-based stages	Different workflows per branch

Use Case	Benefit
Config-driven stages	Change pipeline without editing Jenkinsfile
Dynamic parallel stages	Faster builds via concurrency

5. Step-by-Step Procedures

Step 0: Create the Job

1. Click **New Item** → Name: Dynamic-Stages-Demo → Select **Pipeline** → **OK**

Method 1: Dynamic Stages Using a Groovy List

```
def environments = ['dev', 'qa', 'staging', 'production']

pipeline {
    agent any
    stages {
        stage('Dynamic Deployment') {
            steps {
                script {
                    environments.each { env ->
                        stage("Deploy to ${env}") {
                            echo "Deploying to ${env} environment"
                        }
                    }
                }
            }
        }
    }
}
```

Method 2: Dynamic Stages Based on Git Branch

```
pipeline {
    agent any
    stages {
        stage('Determine Build Type') {
            steps {
                script {
                    def branch = env.BRANCH_NAME ?: 'local'

                    if (branch == 'main') {
                        stage('Production Build') {
                            echo "Building for production..."
                        }
                        stage('Deploy to Production') {
                            echo "Deploying to production..."
                        }
                    } else if (branch.startsWith('feature/')) {
                        stage('Feature Build') {
                            echo "Building feature branch..."
                        }
```

```
                stage('Deploy to Dev') {
                    echo "Deploying to dev environment..."
                }
            } else if (branch.startsWith('hotfix/')) {
                stage('Hotfix Build') {
                    echo "Building hotfix..."
                }
                stage('Deploy to Staging') {
                    echo "Deploying to staging for urgent testing..."
                }
            } else {
                stage('Default Build') {
                    echo "Building for branch: ${branch}"
                }
            }
        }
    }
  }
}
```

Method 3: Dynamic Parallel Stages from a List

```
def testSuites = ['unit', 'integration', 'e2e']

pipeline {
  agent any
  stages {
    stage('Dynamic Parallel Tests') {
      steps {
        script {
          def parallelStages = testSuites.collectEntries {
            ["${it.capitalize()} Tests" : {
              stage("Running ${it} tests") {
                echo "Executing ${it} test suite..."
                sleep 3
                echo "Completed ${it} tests"
              }
            }]
          }
          parallel parallelStages
        }
      }
    }
  }
}
```

Why it works:

- collectEntries builds a map where keys are stage names and values are closures
- parallel parallelStages executes all closures concurrently
- Add a new suite to the list → a new parallel stage appears

Method 4: Dynamic Stages from a YAML Config File

```
def buildConfig

pipeline {
    agent any
    stages {
        stage('Prepare Config') {
            steps {
                script {
                    writeFile file: 'build-config.yaml', text: '''
steps:
  - name: Compile
    command: echo "Compiling source code..."
  - name: Test
    command: echo "Running unit tests..."
  - name: Package
    command: echo "Packaging JAR..."
'''
                    buildConfig = readYaml file: 'build-config.yaml'
                }
            }
        }
        stage('Dynamic Build Stages') {
            steps {
                script {
                    buildConfig.steps.each { step ->
                        stage(step.name) {
                            sh step.command
                        }
                    }
                }
            }
        }
    }
}
```

Step 4.4: Look at the **Stage View** — three dynamic stages (Compile, Test, Package) appear, generated entirely from the YAML file.

Why it works:

- writeFile creates the YAML file (normally this would live in your repo)
- readYaml parses it into a Groovy map
- buildConfig.steps.each { step -> stage(step.name) { sh step.command } } generates stages dynamically

Change the YAML → stages change. No Jenkinsfile edits needed.

Requires: Pipeline Utility Steps plugin for readYaml.

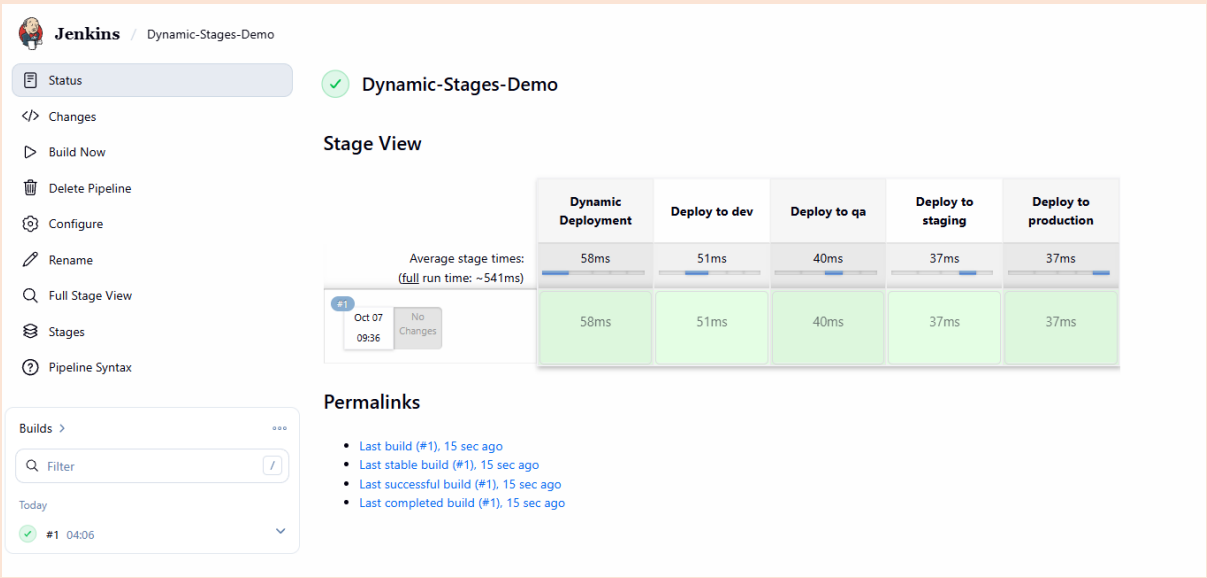
Method 5: Dynamic Parallel Stages from YAML Config

```
def testConfig

pipeline {
    agent any
    stages {
        stage('Prepare Test Config') {
            steps {
                script {
                    writeFile file: 'test-config.yaml', text: '''
tests:
  - name: Unit Tests
    command: echo "Running unit tests..."
  - name: Integration Tests
    command: echo "Running integration tests..."
  - name: UI Tests
    command: echo "Running UI tests..."
'''
                    testConfig = readYaml file: 'test-config.yaml'
                }
            }
        }
        stage('Dynamic Parallel Tests') {
            steps {
                script {
                    def parallelStages = [:]
                    testConfig.tests.each { test ->
                        parallelStages["${test.name}"] = {
                            sh test.command
                        }
                    }
                    parallel parallelStages
                }
            }
        }
    }
}
```

Why this is powerful: A single YAML file controls which tests run, in parallel, with their own commands. Teams can update test coverage without touching the Jenkinsfile.

Implementation:




```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Dynamic-Stages-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Determine Build Type)
[Pipeline] script
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Default Build)
[Pipeline] echo
Building for branch: local
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
[Pipeline] echo
[Unit Tests] Executing unit test suite...
[Pipeline] sleep
[Unit Tests] Sleeping for 3 sec
[Pipeline] echo
[Integration Tests] Executing integration test suite...
[Pipeline] sleep
[Integration Tests] Sleeping for 3 sec
[Pipeline] echo
[E2e Tests] Executing e2e test suite...
[Pipeline] sleep
[E2e Tests] Sleeping for 3 sec
[Pipeline] echo
[Unit Tests] Completed unit tests
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] echo
[Integration Tests] Completed integration tests
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] echo
[E2e Tests] Completed e2e tests
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // parallel
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Dynamic-Stages-Demo

Stage View

	Prepare Config	Dynamic Build Stages	Compile	Test	Package
Average stage times: (full run time: ~1s)	221ms	151ms	346ms	303ms	300ms
#4 Oct 07 09:52 No Changes	221ms	151ms	346ms	303ms	300ms

Permalinks

- Last build (#4), 15 sec ago
- Last stable build (#4), 15 sec ago
- Last successful build (#4), 15 sec ago
- Last completed build (#4), 15 sec ago

```
                    [Pipeline] sh
        [Unit Tests] + echo 'Running unit tests...'
                    Running unit tests...
                    [Pipeline] }
[Integration Tests] + echo 'Running integration tests...'
                    Running integration tests...
                    [Pipeline] }
          [UI Tests] + echo 'Running UI tests...'
                    Running UI tests...
                    [Pipeline] }
                    [Pipeline] // parallel
                    [Pipeline] }
                    [Pipeline] // script
                    [Pipeline] }
                    [Pipeline] // stage
                    [Pipeline] }
                    [Pipeline] // node
                    [Pipeline] End of Pipeline
                    Finished: SUCCESS
```

Conclusion

→Dynamic stages turn static scripts into adaptive, reusable automation. Pipelines use **Groovy loops** for repetitive stages, **conditionals** for branch workflows, and **YAML files** for external configuration. Adding the **parallel** step runs these stages concurrently, significantly cutting build times.
→These methods enable a single Jenkinsfile to handle branch deployments, environment matrices, and dynamic testing. While the **Pipeline Utility Steps** plugin helps read YAML, the core capability is built straight into Jenkins, making pipelines flexible, maintainable, and fast.

Task 14: Strategies for Handling Different Branches in a Jenkinsfile

1. Objective

To make a single Jenkinsfile behave differently based on the Git branch being built, using three complementary strategies: conditional execution (when / if-else), environment variables, and Multibranch Pipeline projects.

2. Purpose

In real-world development workflows, different Git branches serve different purposes and require different pipeline behavior:

Branch Type	Purpose	Needs
main / master	Production-ready code	Strict tests, production deployment
Develop	Feature integration	Full tests, staging deployment
feature/*	New features	Quick build, basic tests, no deploy
hotfix/*	Urgent production fixes	Fast but careful pipeline
release/*	Pre-release preparation	Full tests, pre-production deploy

Without branch-aware pipelines:

- Wasted resources — feature branches trigger production-grade test suites
- Risky deployments — feature branches can accidentally deploy to production
- Poor notifications — wrong team members get alerted
- Duplicated work — maintaining multiple separate jobs for each branch

3. Plugins Needed

Feature	Plugin Required
when { branch 'main' } directive	None — built into Pipeline
if-else in script { }	None — Groovy runtime
env.BRANCH_NAME	None — built into Jenkins
Multibranch Pipeline	Pipeline: Multibranch plugin
Git branch discovery	Git plugin
Branch-specific Jenkinsfiles	None — git feature

4. Concepts Reference

4.1 What Is env.BRANCH_NAME?

A Jenkins environment variable automatically set by Multibranch Pipeline jobs. It contains the branch name being built (e.g., main, develop, feature/login).

In non-multibranch jobs: env.BRANCH_NAME is null.

4.2 Three Strategies Overview

Strategy	Best For	How
Conditional Execution	Simple branch-specific logic	when directive + if-else
Environment Variables	Dynamic values per branch	BRANCH_NAME, custom vars
Multibranch Pipeline	Many branches, unique pipelines	Auto-detect + branch Jenkinsfiles

4.3 Common Branch Patterns

Pattern	Meaning
branch 'main'	Exact match
env.BRANCH_NAME.startsWith('feature/')	Prefix match
env.BRANCH_NAME.contains('hotfix')	Substring match
anyOf { branch 'develop'; branch 'staging' }	OR combination

5. Step-by-Step Procedures

Step 0: Create the Job

1. Click **New Item** → Name: Branch-Strategies-Demo → Select **Pipeline** → **OK**

Important: For testing, we'll manually set env.BRANCH_NAME at the top since this isn't a real Multibranch Pipeline job yet.

Method 1: Conditional Execution with when Directive

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo "Building branch: ${env.BRANCH_NAME ?: 'local'}"
            }
        }
        stage('Deploy to Production') {
            when {
                branch 'main'
            }
            steps {
                echo 'Deploying to production...'
            }
        }
        stage('Deploy to Staging') {
            when {
                anyOf {
                    branch 'develop'
                    branch 'staging'
                }
            }
            steps {
                echo 'Deploying to staging...'
            }
        }
        stage('Feature Build Info') {
            when {
                expression { env.BRANCH_NAME != null && env.BRANCH_NAME.startsWith('feature/') }
            }
            steps {
                echo 'Feature branch detected — no deployment.'
            }
        }
    }
}
```

Why: This job isn't tied to a branch, so env.BRANCH_NAME is null. All conditional stages skip. This is expected behavior.

To test with a real branch value: Modify the script to manually set env.BRANCH_NAME:

```
env.BRANCH_NAME = 'main'  // For testing only

pipeline {
    agent any
    stages {
        stage('Build') {
            steps { echo "Building branch: ${env.BRANCH_NAME}" }
        }
        stage('Deploy to Production') {
            when { branch 'main' }
            steps { echo 'Deploying to production...' }
        }
        stage('Deploy to Staging') {
            when { anyOf { branch 'develop'; branch 'staging' } }
            steps { echo 'Deploying to staging...' }
        }
    }
}
```

Method 2: Conditional Execution with if-else

```
env.BRANCH_NAME = 'feature/login'

pipeline {
    agent any
    stages {
        stage('Build and Test') {
            steps {
                script {
                    if (env.BRANCH_NAME == 'main' || env.BRANCH_NAME == 'develop') {
                        echo 'Running FULL test suite...'
                    } else {
                        echo 'Running QUICK tests...'
                    }
                }
            }
        }
        stage('Deploy') {
            steps {
                script {
                    if (env.BRANCH_NAME == 'main') {
                        echo 'Deploying to PRODUCTION...'
                    } else if (env.BRANCH_NAME == 'develop') {
                        echo 'Deploying to STAGING...'
                    } else {
                        echo "Skipping deployment for branch: ${env.BRANCH_NAME}"
                    }
                }
```

```
        }
      }
    }
}
```

Step 2.4: Change env.BRANCH_NAME = 'main' and rerun. Observe:

text

```
Running FULL test suite...
Deploying to PRODUCTION...
```

Change env.BRANCH_NAME = 'develop' **and rerun. Observe:**

text

```
Running FULL test suite...
Deploying to STAGING...
```

Why this works: One Jenkinsfile, three different behaviors based on branch name.

Method 3: Custom Environment Variables with Ternary Operator

```
env.BRANCH_NAME = 'develop'

pipeline {
    agent any
    environment {
        DEPLOY_TARGET = "${env.BRANCH_NAME == 'main' ? 'production' : 'staging'}"
        ENVIRONMENT_TYPE = "${env.BRANCH_NAME == 'main' ? 'prod' : (env.BRANCH_NAME ==
'develop' ? 'nonprod' : 'dev')}"
    }
    stages {
        stage('Deploy') {
            steps {
                echo "Deploy target: ${env.DEPLOY_TARGET}"
                echo "Environment type: ${env.ENVIRONMENT_TYPE}"
                echo "Deploying to ${env.DEPLOY_TARGET}..."
            }
        }
    }
}
```

Why this works: The ternary operator (? :) picks a value based on the branch. Custom env vars are accessible throughout the pipeline via env.VARIABLE_NAME.

Method 4: Custom Branch Type Variable (Groovy Logic in Environment Block)

```
pipeline {
    agent any
    stages {
        stage('Create JSON') {
            steps {
                sh '''
                cat > build_config.json <<EOF
{
    "projectName": "MyApp",
    "version": "2.5.1",
    "steps": [
```

```
      {"name": "compile"},
      {"name": "test"},
      {"name": "package"}
    ]
}
EOF
            '''
        }
      }
      stage('Parse JSON') {
        steps {
          script {
            // Use Jenkins built-in readJSON step (sandbox-safe)
            def config = readJSON file: 'build_config.json'

            echo "Building ${config.projectName} version ${config.version}"
            config.steps.each { step ->
              echo "Executing step: ${step.name}"
            }
          }
        }
      }
    }
}
```

Why this works: The environment block evaluates the branch name once and stores its **type** in a reusable variable. All downstream stages use BRANCH_TYPE for clean, readable logic.

Method 5: Set Up a Multibranch Pipeline (Real-World)

Step 5.1: Install Required Plugins (if not already installed)

1. Go to **Manage Jenkins → Plugins → Available**
2. Search and install:
    - **Pipeline: Multibranch**
    - **Git** (if not already installed)
3. Restart Jenkins if needed

Step 5.2: Create the Multibranch Pipeline Job

1. Click **New Item**
2. Name: Branch-Strategies-Multibranch
3. Select **Multibranch Pipeline**
4. Click **OK**

Step 5.3: Configure Branch Source

1. Scroll to **Branch Sources**
2. Click **Add source → Git**
3. Enter your repository URL (https://github.com/Sunith3210/jenkins-shared-library.git)
4. Add credentials if required (for private repos)
5. Under **Behaviors**, leave defaults (discover all branches)
6. Click **Save**

Step 5.4: Jenkins Auto-Discovers Branches

Afte

Method 6: Branch-Specific Jenkinsfiles

Example 1 — Feature branch Jenkinsfile

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building feature branch'
            }
        }
        stage('Test') {
            steps {
                echo 'Running quick tests'
            }
        }
    }
}
```

Example 2 — Main branch Jenkinsfile:

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building main branch'
            }
        }
        stage('Test') {
            steps {
                echo 'Running comprehensive test suite'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying to production'
            }
        }
    }
}
```

Method 7: Combined Approach — Base Jenkinsfile with Override

```
pipeline {
    agent any
    environment {
        // Use triple double-quotes """ for multi-line string evaluation in environment blocks
        BRANCH_TYPE = """${
            (env.BRANCH_NAME ?: 'local') == 'main' ? 'production' :
            (env.BRANCH_NAME ?: 'local') == 'develop' ? 'development' :
            (env.BRANCH_NAME ?: 'local').startsWith('feature/') ? 'feature' :
            (env.BRANCH_NAME ?: 'local').startsWith('hotfix/') ? 'hotfix' :
```

```
        'unknown'
    }"""
  }
  stages {
    stage('Build') {
      steps {
        script {
          if (env.BRANCH_TYPE == 'production' || env.BRANCH_TYPE == 'development') {
            echo 'Building with full optimization...'
          } else {
            echo 'Building with quick mode...'
          }
        }
      }
    }
    stage('Test') {
      steps {
        script {
          if (env.BRANCH_TYPE == 'production') {
            echo 'Running comprehensive test suite (200+ tests)...'
          } else if (env.BRANCH_TYPE == 'development') {
            echo 'Running integration tests (100 tests)...'
          } else {
            echo 'Running quick smoke tests (10 tests)...'
          }
        }
      }
    }
    stage('Deploy') {
      when {
        expression { env.BRANCH_TYPE == 'production' || env.BRANCH_TYPE == 'development' }
      }
      steps {
        script {
          if (env.BRANCH_TYPE == 'production') {
            echo 'Deploying to PRODUCTION...'
          } else {
            echo 'Deploying to STAGING...'
          }
        }
      }
    }
  }
}
```

Result: One Jenkinsfile that handles all common branch types. Feature branches skip deployment entirely. Main gets production-grade testing. Develop gets staging deployment

Implementation:

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on jenkins-agent-01 in /home/jenkins/workspace/Branch-Strategies-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building branch: local
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy to Production)
Stage "Deploy to Production" skipped due to when conditional
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy to Staging)
Stage "Deploy to Staging" skipped due to when conditional
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Feature Build Info)
Stage "Feature Build Info" skipped due to when conditional
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on jenkins-agent-01 in /home/jenkins/workspace/Branch-Strategies-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build and Test)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
Running QUICK tests...
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
Skipping deployment for branch: feature/login
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
Started by user admin
[Pipeline] Start of Pipeline (hide)
[Pipeline] node
Running on jenkins-agent-01 in /home/jenkins/workspace/Branch-Strategies-Demo
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploy target: staging
[Pipeline] echo
Environment type: nonprod
[Pipeline] echo
Deploying to staging...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Download progress

Preparation
- Checking internet connectivity
- Checking update center connectivity
- Success

Config File Provider	Success
SSH server	Success
JavaMail API	Success
Pipeline: Multibranch with defaults	Success
Loading plugin extensions	Success

→ Go back to the top page
(you can start using the installed plugins right away)

→ Restart Jenkins when installation is complete and no jobs are running

New Item

Enter an item name

Branch-Strategies-Multibranch

Select an item type

Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

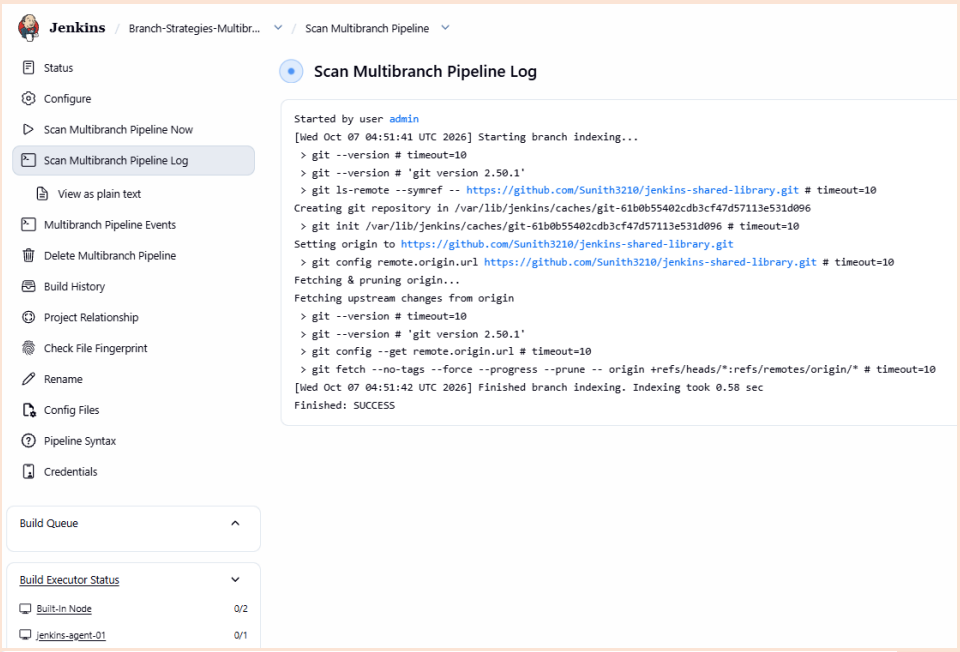
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

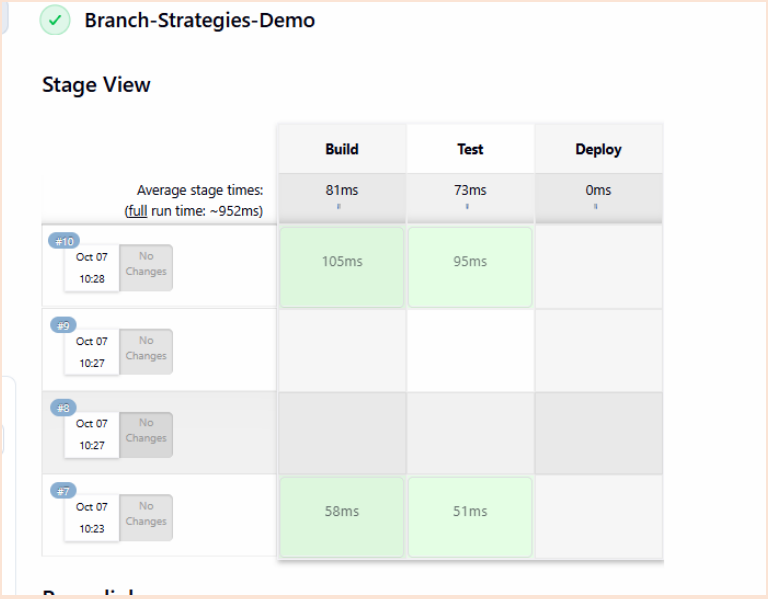
Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.

Jenkins Article-2




```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on jenkins-agent-01 in /home/jenkins/workspace/Branch-Strategies-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building feature branch
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Running quick tests
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Conclusion

→Managing multiple branches in a single Jenkinsfile is vital for modern CI/CD. Pipelines can use when/if-else conditionals for stage gating, environment variables (like the BRANCH_TYPE pattern) for dynamic values, and Multibranch Pipelines for automatic branch discovery.

→Together, these strategies allow a single Jenkinsfile to serve many branches, optimizing resources and ensuring safe deployments. No advanced plugins are needed for logic and variables; you only need the Multibranch Pipeline plugin to enable auto-discovery.

Task 15: Getting a JSON Object as a Parameter in a Jenkins Pipeline

1. Objective

To pass structured JSON data into a Jenkins pipeline using a Multiline String Parameter, then parse and access individual fields inside the pipeline using two methods: the Jenkins-native readJSON step and Groovy's JsonSlurper class.

2. Purpose

Jenkins has no native JSON parameter type. However, many real-world pipelines need to receive structured data as input — environment configs, deployment targets, test matrices, build metadata, or API payloads. JSON is the universal format for this data.

Common scenarios:

- **Deployment matrices** — pass a JSON list of environments and their URLs
- **Build configurations** — pass project name, version, and steps as JSON
- **Test configurations** — define test suites and their parameters
- **API payloads** — prepare JSON to send to external systems
- **Trigger payloads** — accept webhook JSON from other tools
- **Dynamic stage generation** — read a JSON list and generate stages (combines with Task 12)

3. Concepts Reference

3.1 What Is JSON?

JSON (JavaScript Object Notation) is a text format for structured data. It uses key-value pairs and arrays:

```
{
 "name": "Alice",
 "grade": "A",
 "subjects": [
  { "name": "Math", "passed": true },
  { "name": "Science", "passed": false }
 ]
}
```

3.2 Why a Multiline String Parameter?

Jenkins parameter types include String, Boolean, Choice, Password, and Multiline String — but no JSON type. The Multiline String Parameter is the closest fit because it preserves line breaks and formatting, allowing JSON to be entered naturally

3.3 Two Parsing Methods

Method	Type	Best For	Plugin
readJSON	Jenkins pipeline step	Simple JSON access in Declarative pipelines	Pipeline Utility Steps
JsonSlurper	Groovy class	Complex logic, loops, conditionals	None

3.4 Accessing Parsed Data

After parsing:

Syntax	Meaning
myobj.name	Access a top-level field
myobj.subjects[0].name	Access a field in the first array element (0-based)
myobj.subjects.size()	Array length
myobj.subjects.each { ... }	Iterate over array elements

4. Step-by-Step Procedures

Step 0: Create the Job with a JSON Parameter

Step 0.1: Click **New Item** → Name: JSON-Parameter-Demo → Select **Pipeline** → **OK**

Step 0.2: Scroll to the **General** section (top of configuration page)

Step 0.3: Check the box **This project is parameterized**

Step 0.4: Click **Add Parameter** → Select **Multiline String Parameter**

Step 0.5: Fill in:

- **Name:** student
- **Default Value:** (paste this JSON)

```
{
 "name": "Alice",
 "grade": "A",
 "subjects": [
```

```
    { "name": "Math", "passed": true },
    { "name": "Science", "passed": false },
    { "name": "Physics", "passed": true }
  ]
}
```

- **Description:** Student info as JSON

Step 0.6: Scroll to the **Pipeline** section and paste:

```
pipeline {
    agent any
    stages {
        stage("Print JSON Parameter") {
            steps {
                echo "Raw JSON parameter:"
                echo student
            }
        }
    }
}
```

Method 1: Parsing with readJSON

```
pipeline {
    agent any
    stages {
        stage("Parse with readJSON") {
            steps {
                script {
                    // 1. Define the student JSON string properly
                    def student = '''{
                        "name": "Alice",
                        "grade": "A",
                        "subjects": [
                            { "name": "Math", "passed": true },
                            { "name": "Science", "passed": false },
                            { "name": "Physics", "passed": true }
                        ]
                    }'''

                    // 2. Parse using Jenkins built-in readJSON step
                    def myobj = readJSON text: student

                    echo "=========================================="
                    echo "Student Name: ${myobj.name}"
                    echo "Grade: ${myobj.grade}"
                    echo "=========================================="
                    echo "Subjects:"
```

```
            myobj.subjects.each { subject ->
                echo "  - ${subject.name} (passed: ${subject.passed})"
            }
            echo "========================================="
        }
    }
  }
}
```

Verification: Save and run this build. It will successfully parse the JSON string and output Alice's grades and subjects in the console log.

Method 2: Parsing with Groovy

```
pipeline {
    agent any
    stages {
        stage("Parse with readJSON") {
            steps {
                script {
                    // 1. Define the student JSON string properly
                    def student = '''{
                        "name": "Alice",
                        "grade": "A",
                        "subjects": [
                            { "name": "Math", "passed": true },
                            { "name": "Science", "passed": false },
                            { "name": "Physics", "passed": true }
                        ]
                    }'''

                    // 2. Safely parse using Jenkins built-in readJSON step
                    def myobj = readJSON text: student

                    echo "========================================="
                    echo "Student: ${myobj.name}"
                    echo "Grade: ${myobj.grade}"
                    echo "========================================="

                    myobj.subjects.each { subject ->
                        if (subject.passed) {
                            echo "☑ Passed: ${subject.name}"
                        } else {
                            echo "❌ Failed: ${subject.name}"
                        }
                    }
                    echo "========================================="
                }
            }
        }
```

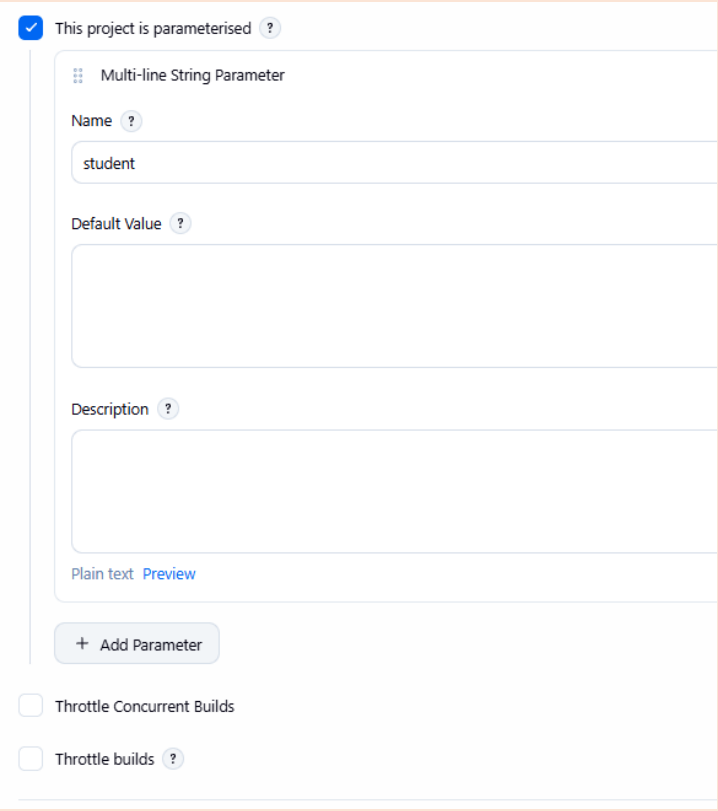
```
    }
}
```

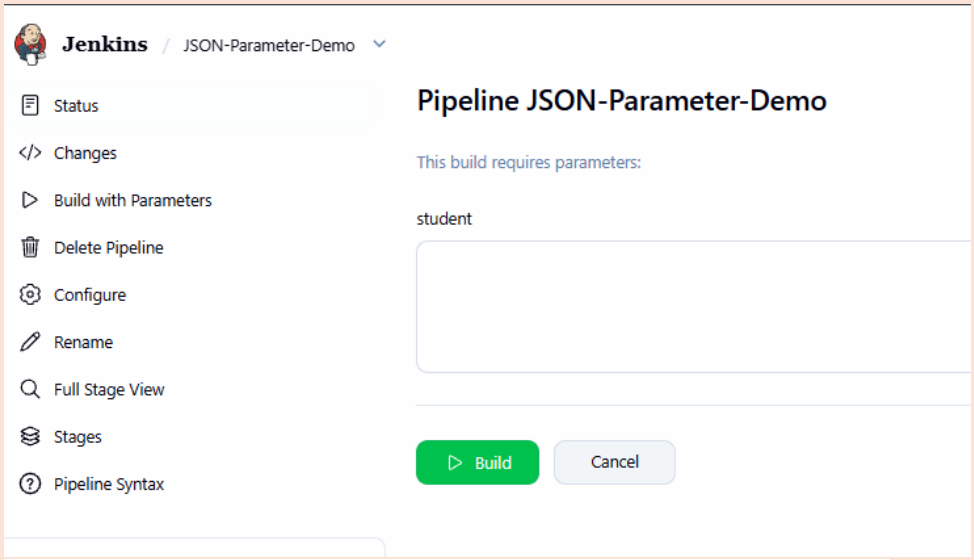
Method 3: Using params.student (Alternative Access)

```
pipeline {
    agent any
    parameters {
        // Define the parameter so params.student is valid
        text(name: 'student', defaultValue: '''{
            "name": "Alice",
            "grade": "A"
        }''', description: 'JSON string for student')
    }
    stages {
        stage("Access via params") {
            steps {
                script {
                    // Safely parse the parameter using Jenkins built-in readJSON
                    def myobj = readJSON text: params.student

                    echo "Name: ${myobj.name}"
                    echo "Grade: ${myobj.grade}"
                }
            }
        }
    }
```

Implementation:






```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/JSON-Parameter-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Parse JSON Parameter)
[Pipeline] script
[Pipeline] {
[Pipeline] readJSON
[Pipeline] echo
Student Name: Alice
[Pipeline] echo
Grade: A
[Pipeline] echo
--- Subjects ---
[Pipeline] echo
Subject: Math - Status: PASSED
[Pipeline] echo
Subject: Science - Status: FAILED
[Pipeline] echo
Subject: Physics - Status: PASSED
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
==========================================
[Pipeline] echo
Student Name: Alice
[Pipeline] echo
Grade: A
[Pipeline] echo
==========================================
[Pipeline] echo
Subjects:
[Pipeline] echo
  - Math (passed: true)
[Pipeline] echo
  - Science (passed: false)
[Pipeline] echo
  - Physics (passed: true)
[Pipeline] echo
==========================================
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/JSON-Parameter-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Parse with readJSON)
[Pipeline] script
[Pipeline] {
[Pipeline] readJSON
[Pipeline] echo
==========================================
[Pipeline] echo
Student: Alice
[Pipeline] echo
Grade: A
[Pipeline] echo
==========================================
[Pipeline] echo
✅ Passed: Math
[Pipeline] echo
❌ Failed: Science
[Pipeline] echo
✅ Passed: Physics
[Pipeline] echo
==========================================
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
[Pipeline] { (Parse with readJSON)
[Pipeline] script
[Pipeline] {
[Pipeline] readJSON
[Pipeline] echo
==========================================
[Pipeline] echo
Student: Alice
[Pipeline] echo
Grade: A
[Pipeline] echo
==========================================
[Pipeline] echo
✅ Passed: Math
[Pipeline] echo
❌ Failed: Science
[Pipeline] echo
✅ Passed: Physics
[Pipeline] echo
==========================================
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Conclusion

In this article, we've covered the basics of working with JSON strings in Jenkins. We showed how to create a Jenkins parameter with JSON text and then parse it using *readJSON* or *JsonSlurper*. The *readJSON* is a pipeline utility step that provides an easy way to parse JSON text in Jenkins declarative pipelines. The *JsonSlurper* is a standard Groovy class that we can import, and it provides different methods to work with JSON text

Task 16: How to Check if a Pipeline Parameter Is Empty in Jenkins

1. Objective

To define parameters in Jenkins pipelines (via GUI and Jenkinsfile), use them dynamically, and safely check whether a parameter value is empty using two methods: Groovy's isEmpty() function and a raw if-else comparison against '' and null.

2. Purpose

Parameters make Jenkins pipelines dynamic — a single pipeline can deploy to dev, staging, or prod based on user input. However, an empty parameter can cause serious problems:

- **Wrong deployments** — an empty environment name might default to production or fail silently
- **Pipeline crashes** — null values cause NullPointerException in string operations
- **Silent failures** — pipeline succeeds but does nothing useful
- **Security issues** — empty credentials or URLs might bypass validation
- **Unclear errors** — hard to debug when the failure comes from missing input

Real-world scenarios where empty parameters matter:

- **Environment selection** — user forgets to choose a target environment
- **Version input** — user leaves version blank and pipeline fails at deploy
- **Credentials** — API keys or tokens left blank cause authentication failures
- **Service names** — dynamic pipelines triggered by other jobs may pass empty parameters
- **Optional fields** — some parameters are optional but must be handled gracefully

3. Concepts Reference

3.1 What Is a Jenkins Parameter?

A parameter is an input value the user provides when triggering the build. Jenkins treats it as both:

- A **variable** named after the parameter (e.g., environment)
- An **environment variable** (env.environment or params.environment)

3.2 Two Ways to Define Parameters

Method	Where	Best For
GUI	Job configuration → "This project is parameterized"	Quick setup, non-technical users
Jenkinsfile	parameters { } block	Version control, code review, portability

Rule: Define a parameter in **one place only** — GUI **or** Jenkinsfile, never both. Defining in both causes conflicts.

3.3 Common Parameter Types

Type	Example Use
String	Environment name, version, URL
Choice	Predefined list (dev/staging/prod)
Boolean	Feature toggle
Password	Credentials
Multiline String	JSON payloads (Task 15)

3.4 Two Ways to Check for Empty

Method	Syntax	Notes
isEmpty()	environment.isEmpty()	Clean, handles null in most cases
Raw comparison	environment == '' || environment == null	Explicit null safety

4. Step-by-Step Procedures

Step 0: Define the Parameter via GUI

Step 0.1: Click **New Item** → Name: Empty-Parameter-Demo → Select **Pipeline** → **OK**

Step 0.2: Scroll to the **General** section

Step 0.3: Check the box **This project is parameterized**

Step 0.4: Click **Add Parameter** → Select **String Parameter**

Step 0.5: Fill in:

- **Name:** environment
- **Default Value:** *(leave empty)*
- **Description:** Target environment for deployment (e.g., dev, staging, prod)

Step 0.6: Scroll to the **Pipeline** section and paste:

```
pipeline {
  agent any
  stages {
    stage("Print Parameter Value") {
      steps {
        echo "We'll deploy on the '${environment}' environment"
      }
    }
  }
}
```

Step 0.8: Click **Build with Parameters** → Enter production in the field → Click **Build**

Step 0.9: Observe the console output:

text

We'll deploy on the 'production' environment

Method 1: Checking with isEmpty()

```
pipeline {
  agent any
  parameters {
    string(
      name: 'environment',
      defaultValue: '',
      description: 'Target environment for deployment'
    )
  }
  stages {
    stage("Validate and Print Parameter") {
      steps {
        script {
          if (environment.isEmpty()) {
            echo "❌ The environment parameter cannot be empty."
            echo "Please provide a value (e.g., dev, staging, prod)."
          } else {
            echo "☑ We'll deploy on the '${environment}' environment."
          }
```

```
            }
        }
    }
  }
}
```

Step 1.4: Observe the console output:

text

❌ The environment parameter cannot be empty.
Please provide a value (e.g., dev, staging, prod).
Finished: SUCCESS

Why it works:

- environment.isEmpty() returns true if the string is empty (or null in Groovy)
- The if-else must be inside a script { } block because it is Groovy code, not a Jenkins step
- The pipeline continues either way — you could replace the empty branch with error "..." to abort

Method 2: Checking with Raw if-else Comparison

```
pipeline {
  agent any
  parameters {
    string(
      name: 'environment',
      defaultValue: '',
      description: 'Target environment for deployment'
    )
  }
  stages {
    stage("Validate with Raw Comparison") {
      steps {
        script {
          if (environment == '' || environment == null) {
            echo "❌ The environment parameter is empty (raw comparison)."
          } else {
            echo "☑ We'll deploy on the '${environment}' environment."
          }
        }
      }
    }
  }
}
```

Step 2.2: Save → Build with Parameters → leave empty → Build

Step 2.3: Observe the console output:

text

❌ The environment parameter is empty (raw comparison).
Finished: SUCCESS

Step 2.4: Run again with prod:

text
✅ We'll deploy on the 'prod' environment.
Finished: SUCCESS

Why this works:

- environment == '' catches empty strings
- environment == null catches unset/null values
- The || (OR) means either condition triggers the empty branch
- More explicit than isEmpty(), safer when null is a real possibility

Method 3: Aborting the Pipeline on Empty Parameter

```
pipeline {
  agent any
  parameters {
    string(
      name: 'environment',
      defaultValue: '',
      description: 'Target environment for deployment'
    )
  }
  stages {
    stage("Validate") {
      steps {
        script {
          if (!environment?.trim()) {
            error "❌ Environment parameter is required. Aborting pipeline."
          }
          echo "☑ Deploying to '${environment}'."
        }
      }
    }
    stage("Deploy") {
      steps {
        echo "🚀 Deploying to '${environment}'..."
      }
    }
  }
}
```

Run with empty parameter → Observe:

text
ERROR: ❌ Environment parameter is required. Aborting pipeline.
Finished: FAILURE

Run with staging **→ Observe:**

text
✅ Deploying to 'staging'.
🚀 Deploying to 'staging'...
Finished: SUCCESS

Why this is better than just printing:

- error aborts the pipeline — Deploy stage never runs
- Prevents accidental deployment with a missing environment
- Uses ?.trim() to also catch whitespace-only values (e.g., user typed only spaces)

Method 4: Providing a Default Value When Empty

```
pipeline {
  agent any
  parameters {
    string(
      name: 'environment',
      defaultValue: '',
      description: 'Target environment for deployment'
    )
  }
  stages {
    stage("Apply Default and Deploy") {
      steps {
        script {
          def target = environment?.trim() ?: 'dev'
          echo "Target environment: ${target}"
          echo "🚀 Deploying to '${target}'..."
        }
      }
    }
  }
}
```

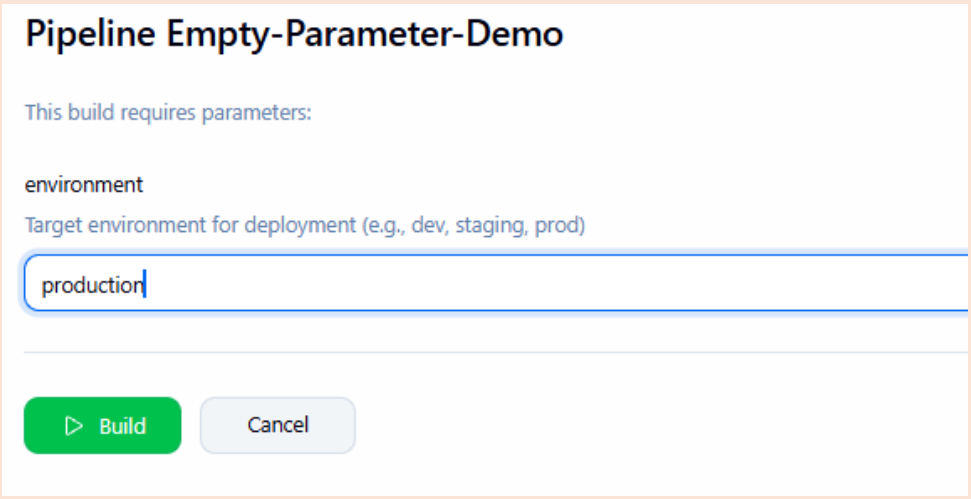
Run with empty parameter → Observe:

text
Target environment: dev
🚀 Deploying to 'dev'...
Finished: SUCCESS

Run with staging **→ Observe:**

text
Target environment: staging
🚀 Deploying to 'staging'...
Finished: SUCCESS

Implementation:



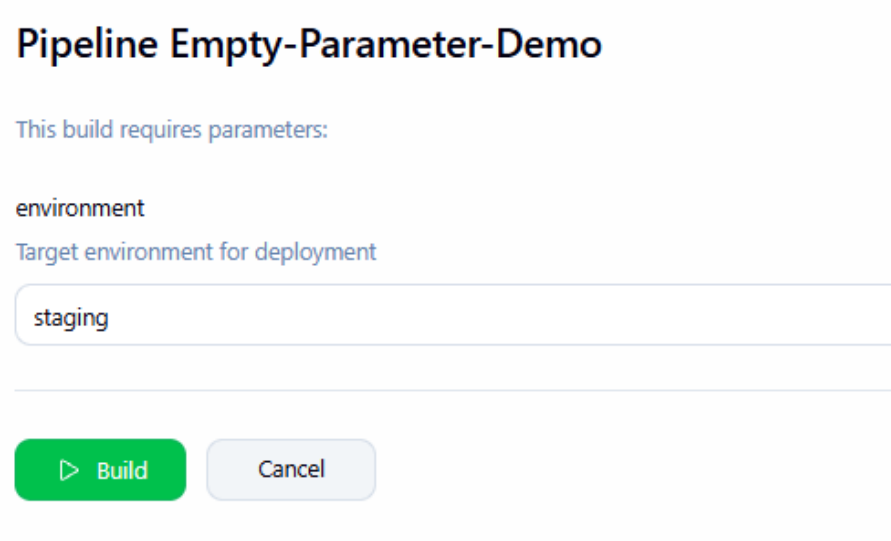

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Empty-Parameter-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Print Parameter Value)
[Pipeline] echo
We'll deploy on the 'production' environment
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Empty-Parameter-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Validate and Print Parameter)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
❌ The environment parameter cannot be empty.
[Pipeline] echo
Please provide a value (e.g., dev, staging, prod).
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Empty-Parameter-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Validate with Raw Comparison)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
❌ The environment parameter is empty (raw comparison).
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Empty-Parameter-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Validate with Raw Comparison)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
✅ We'll deploy on the 'prod' environment.
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Empty-Parameter-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Validate)
[Pipeline] script
[Pipeline] {
[Pipeline] error
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
Stage "Deploy" skipped due to earlier failure(s)
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: ❌ Environment parameter is required. Aborting pipeline.
Finished: FAILURE
```




```
Started by user admin
[Pipeline] Start of Pipeline (hide)
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Empty-Parameter-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Validate)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
✅ Deploying to 'staging'.
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
🚀 Deploying to 'staging'...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Conclusion

This article covered Jenkins parameter basics and how to check if a value is empty. We explored defining parameters via both the **UI configuration** and **declarative Jenkinsfile scripts**. Finally, we demonstrated how to verify empty inputs using the **isEmpty() string function** or a standard **if-else statement**

Task 17: Jenkins Walkthrough and Scripting Filesystem Operations

1. Objective

To understand Jenkins core terminology (jobs, pipelines, stages, steps, nodes, executors, builds), explore the deployment structure (Web UI, CLI, logs, security), and implement filesystem operations (copy and move files) using three methods: shell commands, native Groovy scripts, and the File Operations plugin.

2. Purpose

File movement and duplication are among the most common tasks automated by Jenkins, appearing in scenarios like:

- **Package installation** — moving built artifacts to deployment directories
- **Container deployment** — copying configuration files into container build contexts
- **General configuration** — distributing config files to multiple targets
- **Data backup** — duplicating important files to backup locations
- **Artifact archival** — moving build outputs to storage
- **Log rotation** — moving old logs out of active directories

3. Concepts Reference

3.1 Jenkins Terminology

Term	Meaning
Job (Project)	Top-level unit of execution in Jenkins
Pipeline	A job type that groups tasks, written in Groovy (Jenkinsfile)
Stage	A phase in the pipeline (e.g., Build, Test, Deploy)
Step	A single task inside a stage
Node / Agent	A machine that runs steps
Executor	A worker on a node that runs a single step at a time
Build	One execution of a job or step
Workspace	Directory where a job's files live ($JENKINS_HOME/workspace/<job>)
Plugin	Extension that adds functionality

3.2 Build Statuses

Status	Meaning
aborted	Manually interrupted or timed out
failed	Unrecoverable error
stable	Stable success
successful	Success
unstable	Non-fatal errors

3.3 Key Environment Variables

Variable	Meaning
$JENKINS_HOME	Main Jenkins installation directory
$JENKINS_VERSION	Jenkins version
$JENKINS_SLAVE_AGENT_PORT	Port for agent connections

3.4 Important Subdirectories under $JENKINS_HOME

text

```
/var/jenkins_home/
 ├── jobs/        ← all jobs and their builds
 ├── logs/        ← system logs
 ├── nodes/       ← agent node configurations
 ├── plugins/     ← installed plugins
 ├── secrets/     ← security credentials
 ├── userContent/ ← user-uploaded files
 ├── users/       ← user accounts
 ├── war/         ← Jenkins webapp files
 └── workspace/   ← where jobs run
```

3.5 Groovy Sandbox

By default, Jenkins Pipeline scripts run in a **sandboxed environment**:

- ❌ No direct filesystem access outside the workspace
- ❌ No raw Jenkins API calls
- ✅ Filesystem access to workspace only

To allow unrestricted file access:

1. Uncheck **Use Groovy Sandbox** in the pipeline configuration
2. Script approval by an admin may be required

3.6 Permission Model

Before copying or moving files, verify:

1. **Ownership** — ls -ld /path/to/dir shows owner and permissions
2. **Jenkins user** — usually jenkins (check via **Manage Jenkins → System Information**)
3. **Fix ownership if needed** — chown --recursive jenkins:jenkins /path/

4. Step-by-Step Procedures

Step 0: Set Up Source and Destination Directories

Step 0.1: Open a terminal on the Jenkins host

Step 0.2: Create source and destination directories:

```
sudo mkdir -p /var/source /var/destination
```

Step 0.3: Create a test file in the source directory:

```
echo "This is the test file content." | sudo tee /var/source/file
```

Step 0.4: Fix permissions so the jenkins user can write to both directories:

```
sudo chown --recursive jenkins:jenkins /var/source /var/destination
```

Step 0.5: Verify the setup:

```
→ ls -la /var/source/
→ ls -la /var/destination/
```

Method 1: Copy Files Using Shell Commands (sh / bat)

Step 1.1: Create a new job

1. Click **New Item** → Name: Filesystem-Copy-Demo → Select **Pipeline** → **OK**

```
pipeline {
  agent any // Runs on any available agent
  stages {
    stage('Create file in workspace') {
      steps {
        sh 'echo "This is the test file content." > file.txt'
```

```
        }
    }
    stage('Copy within workspace') {
        steps {
            sh '''
            mkdir -p destination_dir
            cp file.txt destination_dir/
            '''
        }
    }
    stage('Verify Copy') {
        steps {
            sh 'ls -la destination_dir/'
        }
    }
  }
}
```

Method 2: Copy Files Using Native Groovy Script

```
pipeline {
  agent any
  stages {
    stage('Workspace Copy') {
      steps {
        sh '''
        # Create directories inside the local workspace
        mkdir -p source_dir destination_dir

        # Create a test file
        echo "This is the test file content." > source_dir/file

        # Copy file within the workspace
        cp source_dir/file destination_dir/

        echo "===== Verification ====="
        ls -la destination_dir/
        '''
      }
    }
  }
}
```

Method 3: Copy Files Using the File Operations Plugin

Step 3.1: Install the File Operations Plugin

1. Go to **Manage Jenkins → Plugins**
2. Click the **Available** tab
3. Search for File Operations
4. Check the box next to **File Operations Plugin**
5. Click **Install** (or **Install without restart**)
6. Wait for installation to complete

Step 3.2: Create a Pipeline Job

1. Click **New Item** → Name: Filesystem-Copy-Plugin → Select **Pipeline** → **OK**

```
pipeline {
  agent any
  stages {
    stage('plugincopy') {
      steps {
        script {
          dir('/var/source/') {
            fileOperations([
              fileCopyOperation(
                excludes: '',
                flattenFiles: true,
                includes: 'file',
                targetLocation: '/var/destination/'
              )
            ])
          }
        }
      }
    }
    stage('Verify Copy') {
      steps {
        script {
          sh 'ls -la /var/destination/'
        }
      }
    }
  }
}
```

Why this works:

- dir('/var/source/') changes the current working directory
- fileOperations([...]) runs the plugin's file operations
- fileCopyOperation(...) specifies:
  - includes: 'file' — the file pattern to copy
  - targetLocation: '/var/destination/' — destination directory
  - flattenFiles: true — do not preserve directory structure
  - excludes: '' — no exclusions

Method 4: Verify Permissions Before Operations

```
pipeline {
    agent any
    stages {
        stage('Check Permissions') {
            steps {
                script {
                    echo "Source directory:"
                    sh 'ls -ld /var/source/'
                    echo "Destination directory:"
                    sh 'ls -ld /var/destination/'
                    echo "Current user:"
                    sh 'whoami'
                }
            }
        }
    }
}
```

Method 5: Check Jenkins Logs from the Filesystem

Step 5.1: On the Jenkins host, locate the build log for a job:

bash

```
cat $JENKINS_HOME/jobs/Filesystem-Copy-Demo/builds/1/log
```

Expected output: Same as the console output from the Web UI.

Step 5.2: Browse the jobs directory:

bash

```
ls -la $JENKINS_HOME/jobs/
tree $JENKINS_HOME/jobs/Filesystem-Copy-Demo/
```

Expected structure:

text

```
$JENKINS_HOME/jobs/Filesystem-Copy-Demo/
├── builds/
│   ├── 1/
│   │   ├── build.xml
│   │   ├── log
│   │   └── log-index
│   └── permalinks
├── config.xml
└── nextBuildNumber
```

Method 6: Use Jenkins CLI for Build Information

Step 6.1: Download jenkins-cli.jar:

bash

```
wget http://localhost:8080/jnlpJars/jenkins-cli.jar
```

Step 6.2: Generate an API token (see Task 10) or use your password.

Step 6.3: List builds and retrieve console output:

bash

```
# List jobs
```

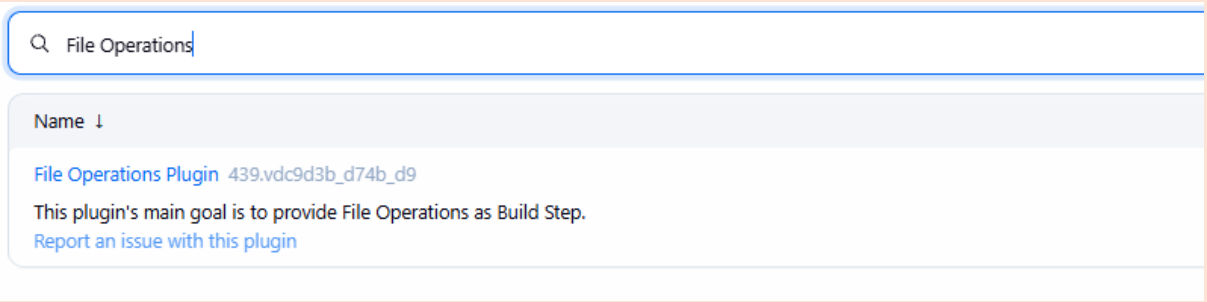
java -jar jenkins-cli.jar -s http://localhost:8080 -auth admin:YOUR_TOKEN list-jobs

Get console output of the last build
java -jar jenkins-cli.jar -s http://localhost:8080 -auth admin:YOUR_TOKEN console Filesystem-Copy-Demo

Implementation:

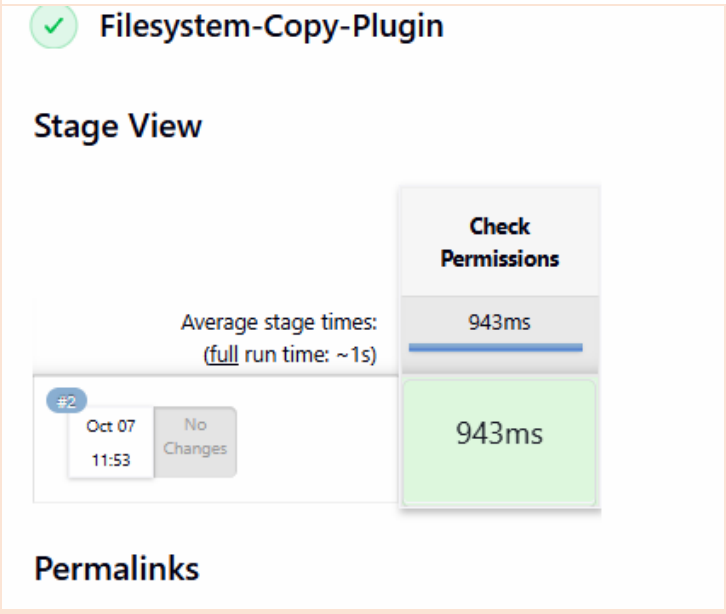
```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on jenkins-agent-01 in /home/jenkins/workspace/Filesystem-Copy-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Create file in workspace)
[Pipeline] sh
+ echo 'This is the test file content.'
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Copy within workspace)
[Pipeline] sh
+ mkdir -p destination_dir
+ cp file.txt destination_dir/
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Verify Copy)
[Pipeline] sh
+ ls -la destination_dir/
total 4
drwxr-xr-x. 2 jenkins jenkins 22 Oct  7 06:09 .
drwxr-xr-x. 3 jenkins jenkins 45 Oct  7 06:09 ..
-rw-r--r--. 1 jenkins jenkins 31 Oct  7 06:09 file.txt
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

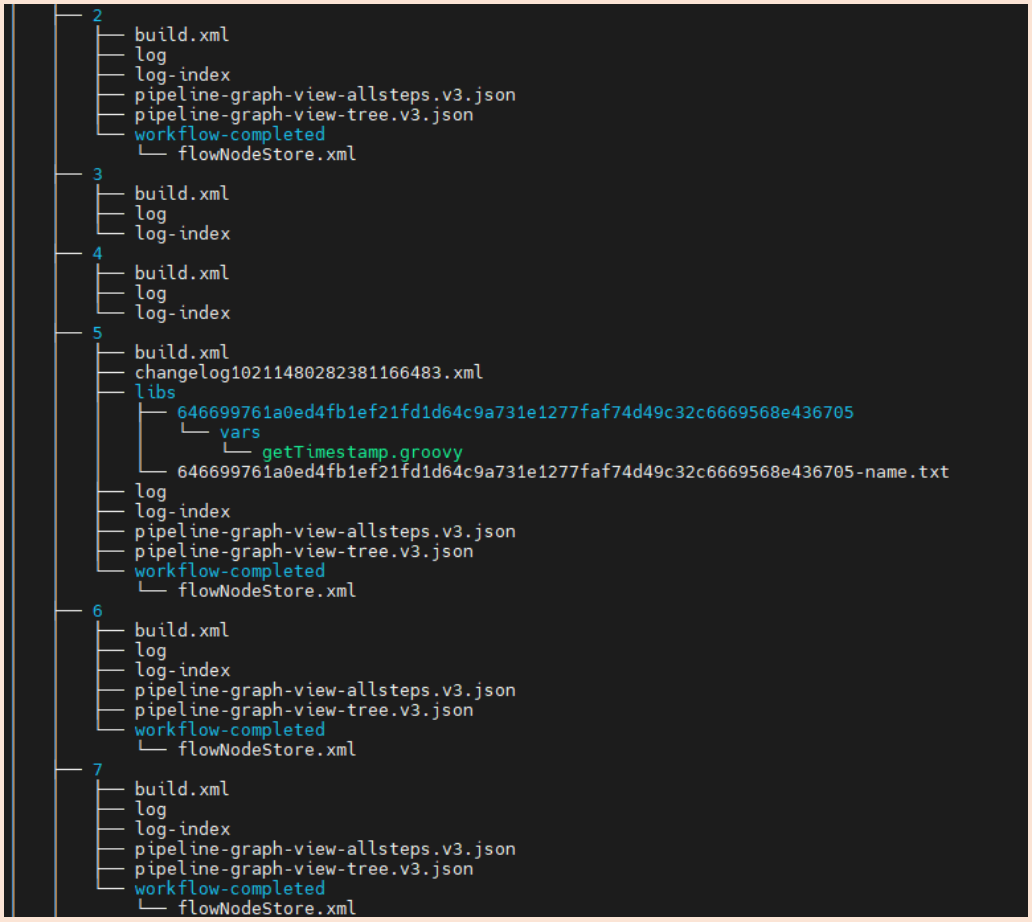
```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on jenkins-agent-01 in /home/jenkins/workspace/Filesystem-Copy-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Workspace Copy)
[Pipeline] sh
+ mkdir -p source_dir destination_dir
+ echo 'This is the test file content.'
+ cp source_dir/file destination_dir/
+ echo '===== Verification ====='
===== Verification =====
+ ls -la destination_dir/
total 8
drwxr-xr-x. 2 jenkins jenkins 34 Oct  7 06:17 .
drwxr-xr-x. 4 jenkins jenkins 63 Oct  7 06:17 ..
-rw-r--r--. 1 jenkins jenkins 31 Oct  7 06:17 file
-rw-r--r--. 1 jenkins jenkins 31 Oct  7 06:09 file.txt
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```




```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node (hide)
Running on Jenkins in /var/lib/jenkins/workspace/Filesystem-Copy-Plugin
[Pipeline] {
[Pipeline] stage
[Pipeline] { (plugincopy)
[Pipeline] script
[Pipeline] {
[Pipeline] dir
Running in /var/source
[Pipeline] {
[Pipeline] fileOperations
File Copy Operation:
/var/source/file
[Pipeline] }
[Pipeline] // dir
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Verify Copy)
[Pipeline] script
[Pipeline] {
[Pipeline] sh
+ ls -la /var/destination/
total 20
drwxr-xr-x.  2 jenkins jenkins    18 Oct  7 06:20 .
drwxr-xr-x. 21 root    root    16384 Oct  7 06:01 ..
-rw-r--r--.  1 jenkins jenkins    31 Oct  7 06:20 file
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Filesystem-Copy-Plugin
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Check Permissions)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
Source directory:
[Pipeline] sh
+ ls -ld /var/source/
drwxr-xr-x. 2 jenkins jenkins 18 Oct  7 06:02 /var/source/
[Pipeline] echo
Destination directory:
[Pipeline] sh
+ ls -ld /var/destination/
drwxr-xr-x. 2 jenkins jenkins 18 Oct  7 06:20 /var/destination/
[Pipeline] echo
Current user:
[Pipeline] sh
+ whoami
jenkins
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```





Summary

In this article, we explored Jenkins with a basic scenario of file moving and copying.

In conclusion, Jenkins is a multi-faceted system that can serve to automate basic tasks such as file copying, but also incorporate these into complex pipelines.

Task 18: Jenkins Pipeline – Git Checkout With Credentials

1. Objective

To create credentials in Jenkins (Username with Password and SSH Username with Private Key) and use them in a pipeline to authenticate and check out code from a private Git repository.

2. Purpose

Modern CI/CD pipelines must clone source code from private Git repositories — GitHub, GitLab, Bitbucket, Azure DevOps, or self-hosted. Public repositories can be cloned anonymously, but private ones require authentication. Jenkins provides a secure credential store so pipelines can access these repos without exposing passwords or private keys in scripts.

Common scenarios where credentials are essential:

- **Private repository checkouts** — proprietary source code
- **Push-back workflows** — committing version bumps or build metadata
- **Protected branch access** — requiring elevated auth
- **Submodule cloning** — pulling dependencies from private repos
- **Multi-repo builds** — pulling code from several repos
- **Webhook-triggered builds** — responding to push events with authenticated pulls

3. Concepts Reference

3.1 What Are Jenkins Credentials?

Credentials in Jenkins are securely stored secrets (passwords, tokens, SSH keys) that pipelines can reference by ID without exposing the actual value in scripts or logs. Jenkins encrypts them at rest and masks them in console output.

3.2 Two Common Credential Types

Type	Best For	URL Format
Username with password	HTTPS Git URLs	https://github.com/user/repo.git
SSH Username with private key	SSH Git URLs	git@github.com:user/repo.git

3.3 Credential ID

A user-defined or auto-generated identifier for a credential. **Always set a memorable ID** (e.g., github-creds) so it can be referenced in pipelines as credentialsId: 'github-creds'.

3.4 Credential Scope

Scope	Meaning
Global	Available to all jobs and pipelines
System	Available only for Jenkins-internal operations

3.5 Host Key Verification (SSH)

When using SSH credentials, Jenkins must trust the remote Git host. The host's public key fingerprint must be:

- Added to Jenkins' known_hosts file, **or**
- Accepted via a strategy like **"Accept first connection"**

If not configured, the git checkout fails with Host key verification failed.

4. Step-by-Step Procedures

Step 0: Verify the Git Plugin Is Installed

1. Go to **Manage Jenkins → Plugins → Installed**
2. Search for Git Plugin
3. Confirm it is installed and enabled

If not installed:

1. Go to **Available** tab
2. Search Git Plugin
3. Install and restart if needed

Method 1: Creating a Username + Password Credential

Step 1.1: Navigate to the Credentials Screen

1. Click **Manage Jenkins** (left sidebar)
2. Under the **Security** section, click **Credentials**
3. Under **Stores scoped to Jenkins**, click **System**
4. Click **Global credentials (unrestricted)**
5. Click **+ Add Credentials** (top-right)

Step 1.2: Fill in the Credential Form

Field	Value
Kind	Username with password
Scope	Global
Username	Your Git username (e.g., myusername)
Password	Your Personal Access Token (not your real password)
ID	github-creds (memorable ID)
Description	GitHub access for pipeline demo

Step 1.3: Click Create

Step 1.4: Verify

You will see the credential listed on the Global credentials page with the ID github-creds. The actual password is not visible.

How to Generate a GitHub Personal Access Token (PAT):

1. Log in to GitHub
2. Click your profile picture → **Settings**
3. Scroll to **Developer settings** → **Personal access tokens** → **Tokens (classic)**
4. Click **Generate new token (classic)**
5. Give it a name, select scopes (repo for private repos), click **Generate token**
6. **Copy the token immediately** — you won't see it again

Method 2: Creating an SSH Username + Private Key Credential

Step 2.1: Generate SSH Keys on the Jenkins Host

Open a terminal on the Jenkins host:

```
ssh-keygen -t ed25519 -C "jenkins@ci"
```

Press Enter to accept defaults (or specify a path). Press Enter twice for no passphrase (or set one and remember it).

This creates:

- **Private key:** ~/.ssh/id_ed25519
- **Public key:** ~/.ssh/id_ed25519.pub

Step 2.2: Add the Public Key to Your Git Host

For GitHub:

1. Log in to GitHub
2. Profile picture → **Settings** → **SSH and GPG keys**
3. Click **New SSH key**
4. Title: Jenkins CI
5. Key type: Authentication Key
6. Paste the content of ~/.ssh/id_ed25519.pub
7. Click **Add SSH key**

Step 2.3: Create the SSH Credential in Jenkins

1. Navigate back to the Credentials screen (Manage Jenkins → Credentials → System → Global credentials)
2. Click **+ Add Credentials**
3. Fill in:

Field	Value
Kind	SSH Username with private key
Scope	Global
ID	github-ssh

Description	SSH key for GitHub
Username	git (or your Git SSH username)
Private Key	Select Enter directly → click Add → paste the private key content
Passphrase	Leave empty (or enter if you set one)

Click **Create**

Step 2.4: Verify

The credential github-ssh now appears in the Global credentials list.

Method 3: Git Checkout Using Username + Password Credential

Step 3.1: Create a Pipeline Job

1. Click **New Item** → Name: Git-Checkout-Demo → Select **Pipeline** → **OK**

Step 3.2: Paste the Pipeline Script

```
pipeline {
  agent any
  stages {
    stage('Git Checkout') {
      steps {
        script {
          git branch: 'main',
            credentialsId: 'github-ssh',
            url: 'https://github.com/Sunith3210/jenkins-shared-library.git'
        }
      }
    }
    stage('Verify Checkout') {
      steps {
        sh 'ls -la'
        sh 'git log -1 --oneline'
      }
    }
  }
}
```

Step 3.4: Observe the Console Output

text

```
Cloning repository https://github.com/your-username/your-repo.git
 > git init /var/lib/jenkins/workspace/Git-Checkout-Demo
Fetching upstream changes from https://github.com/your-username/your-repo.git
 > git fetch --tags --force --progress -- ... +refs/heads/*:refs/remotes/origin/*
 > git checkout -f abc1234
+ ls -la
total 12
drwxr-xr-x ... .
drwxr-xr-x ... ..
-rw-r--r-- ... README.md
-rw-r--r-- ... Jenkinsfile
+ git log -1 --oneline
abc1234 Initial commit
Finished: SUCCESS
```

Method 4: Git Checkout Using SSH Credential

Step 4.1: Update the Pipeline Script

Change the URL to the SSH format and use the SSH credential:

```
pipeline {
    agent any
    stages {
        stage('Git Checkout via SSH') {
            steps {
                script {
                    git branch: 'main',
                        credentialsId: 'github-ssh',
                        url: 'git@github.com:your-username/your-repo.git'
                }
            }
        }
        stage('Verify Checkout') {
            steps {
                sh 'ls -la'
                sh 'git log -1 --oneline'
            }
        }
    }
}
```

Option A — Add host to known_hosts:

On the Jenkins host:

```
ssh-keyscan github.com >> ~/.ssh/known_hosts
```

Ensure the ~/.ssh/known_hosts file is readable by the jenkins user:

bash

```
sudo chown jenkins:jenkins ~/.ssh/known_hosts
```

Option B — Configure Git Plugin to accept first connection:

1. **Go to Manage Jenkins → System**
2. Scroll to **Git Host Key Verification Configuration**
3. Select **Accept first connection** (or **No verification** for testing only)
4. Save
5. Rerun the job

Step 4.5: Verify Success

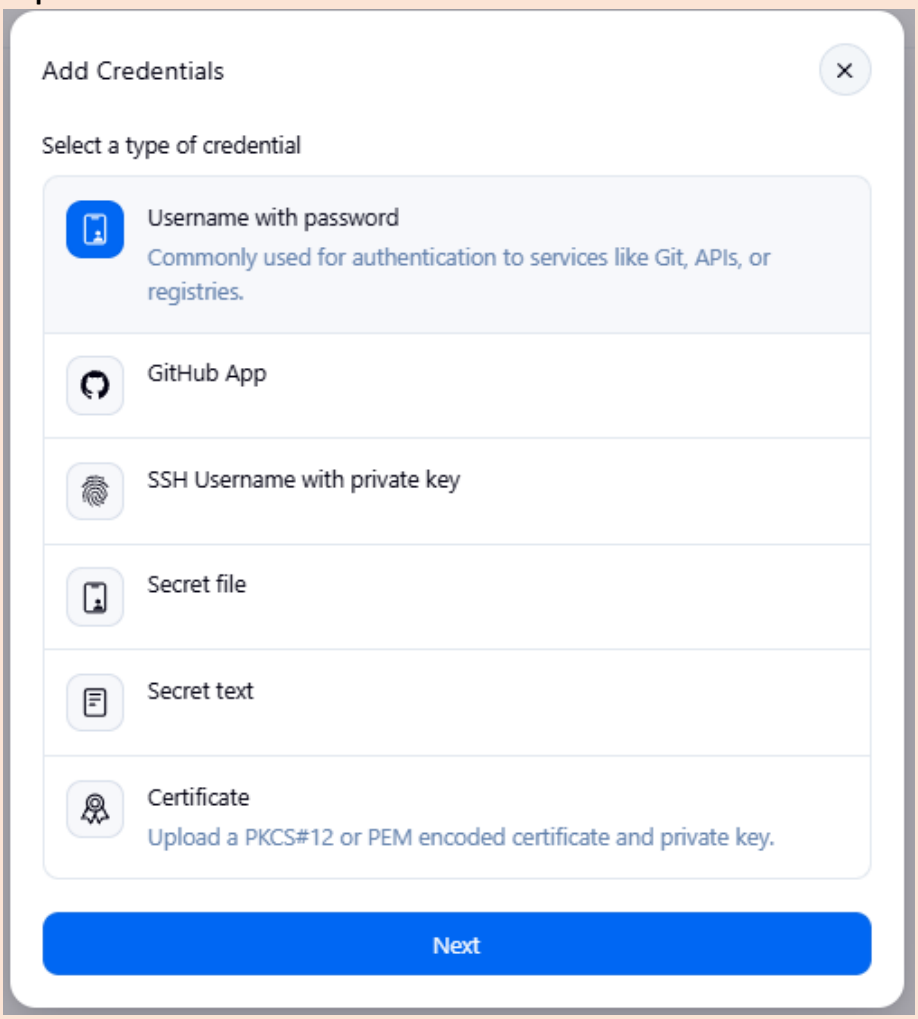
text

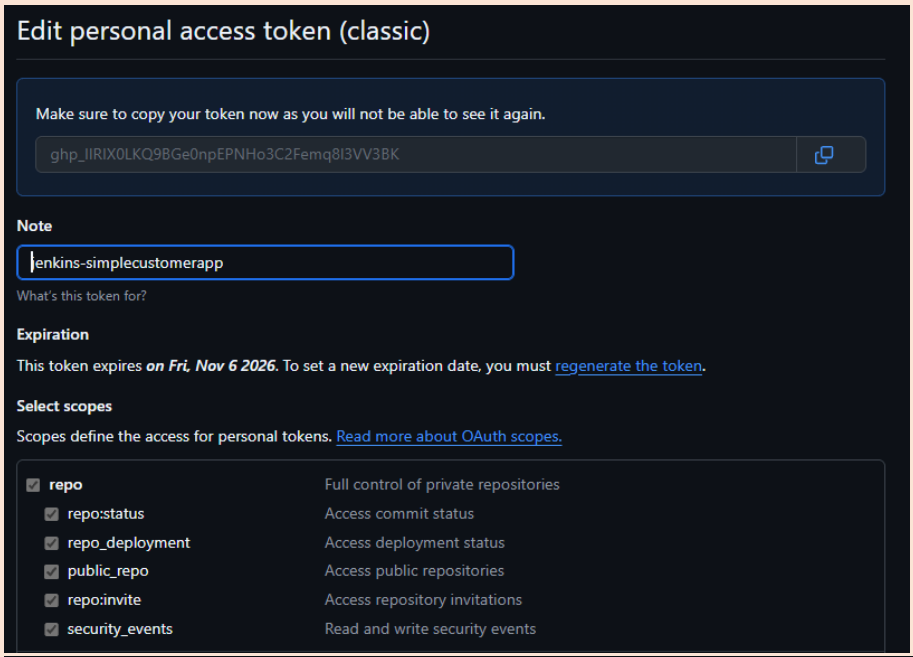
```
Cloning repository git@github.com:your-username/your-repo.git
 > git init /var/lib/jenkins/workspace/Git-Checkout-Demo
...
Finished: SUCCESS
```

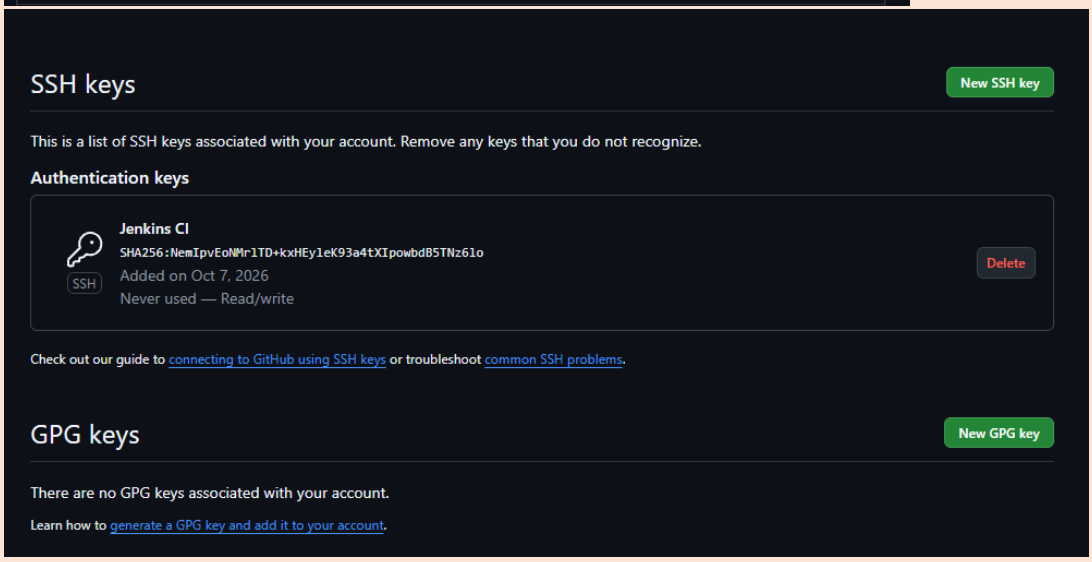
Comparison Table of Credential Types

Aspect	Username + Password	SSH Username + Private Key
URL Format	https://...	git@host:user/repo.git
Setup Complexity	Low (just generate a token)	Medium (generate keys, configure host)
Host Key Verification	Not required	Required
Portability	Works through HTTPS proxies	May be blocked by firewalls
Best For	Most CI/CD workflows	Environments requiring SSH
Token/Key Lifetime	Revocable	Revocable (regenerate)

Implementation:




Edit personal access token (classic)
Make sure to copy your token now as you will not be able to see it again.
ghp_IIRIX0LKQ9BGe0npEPNHo3C2Femq8I3VV3BK
Note
jenkins-simplecustomerapp
What's this token for?
Expiration
This token expires on Fri, Nov 6 2026. To set a new expiration date, you must regenerate the token.
Select scopes
Scopes define the access for personal tokens. Read more about OAuth scopes.
repo
Full control of private repositories
repo:status
Access commit status
repo_deployment
Access deployment status
public_repo
Access public repositories
repo:invite
Access repository invitations
security_events
Read and write security events


SSH keys
New SSH key
This is a list of SSH keys associated with your account. Remove any keys that you do not recognize.
Authentication keys
Jenkins CI
SHA256:NemIpvEoNMrlTD+kxHEyleK93a4tXIpowbdB5TNz6lo
Added on Oct 7, 2026
Never used — Read/write
SSH
Delete
Check out our guide to connecting to GitHub using SSH keys or troubleshoot common SSH problems.
GPG keys
New GPG key
There are no GPG keys associated with your account.
Learn how to generate a GPG key and add it to your account.

```
[Pipeline] git
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
using credential github-ssh
Fetching changes from the remote Git repository
Checking out Revision 6a47491060818072d480774d6559a1c4f2f37f99 (refs/remotes/origin/main)
 > git rev-parse --resolve-git-dir /home/jenkins/workspace/Git-Checkout-Demo/.git # timeout=10
 > git config remote.origin.url https://github.com/Sunith3210/jenkins-shared-library.git # timeout=10
Fetching upstream changes from https://github.com/Sunith3210/jenkins-shared-library.git
 > git --version # timeout=10
 > git --version # 'git version 2.50.1'
using GIT_SSH to set credentials github-ssh
Verifying host key using known hosts file
 > git fetch --tags --force --progress -- https://github.com/Sunith3210/jenkins-shared-library.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 6a47491060818072d480774d6559a1c4f2f37f99 # timeout=10
 > git branch -a -v --no-abbrev # timeout=10
 > git checkout -b main 6a47491060818072d480774d6559a1c4f2f37f99 # timeout=10
Commit message: "Add getTimestamp global function"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Verify Checkout)
[Pipeline] sh
+ ls -la
total 16
drwxr-xr-x.  4 jenkins jenkins    30 Oct  7 07:39 .
drwxr-xr-x. 17 jenkins jenkins 16384 Oct  7 07:16 ..
drwxr-xr-x.  7 jenkins jenkins   146 Oct  7 07:39 .git
drwxr-xr-x.  2 jenkins jenkins    33 Oct  7 07:39 vars
[Pipeline] sh
+ git log -1 --oneline
6a47491 Add getTimestamp global function
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
```

Conclusion

In this article, we went over how to create Jenkins credentials. Then, we talked about how to do a *git checkout* using SSH and username-password credentials.

While demonstrating how to do a *git checkout* with SSH credentials, we emphasized the need to correctly configure the host key verification strategy to ensure success.

Task 19: How to Use Conditional Constructs in Jenkins Pipeline

1. Objective

To implement conditional logic in Jenkins pipelines using five constructs: shell test command, shell if-else, Groovy if-else in a script block, the Groovy ternary operator, and the declarative when directive — across both Declarative and Scripted pipeline styles.

2. Purpose

Without conditionals, every pipeline run executes identically. Real-world CI/CD requires branching logic based on:

- **Environment variables** — skip builds when nothing changed
- **Git branch** — deploy only from main
- **Previous stage results** — run cleanup only on failure
- **Parameters** — conditionally skip expensive tests
- **External state** — check if a service is available before deploying
- **Time-based rules** — run extra tests only on weekdays
- **Build metadata** — run additional checks on release versions

3. Concepts Reference

3.1 The Five Conditional Constructs

#	Construct	Style	Best For
1	sh with test command	Declarative	Simple shell-level checks
2	sh with shell if-else	Declarative	Complex shell logic
3	Groovy if-else in script { }	Declarative	Most common logic
4	Groovy ternary operator	Declarative	One-liner either/or
5	when directive	Declarative	Skip entire stages

3.2 Declarative vs Scripted

Aspect	Declarative	Scripted
Structure	pipeline { } block	node { } block
Groovy code	Must be inside script { }	Anywhere in the block
Readability	Higher	Lower (more code-like)
Flexibility	Limited	High

3.3 Environment Variables in Conditions

Variable	Meaning
env.SKIP_BUILD	Custom variable checked in examples
env.BRANCH_NAME	Current Git branch (in Multibranch)
env.BUILD_NUMBER	Current build number
env.JOB_NAME	Name of the job

3.4 The when Directive Sub-Conditions

Condition	Meaning
branch 'main'	Only on main branch
expression { ... }	Custom Groovy expression
environment name: 'X', value: 'Y'	Env var matches
not { ... }	Invert
anyOf { ... }	True if ANY match
allOf { ... }	True if ALL match

4. Step-by-Step Procedures

Step 0: Create the Jobs

For each method, create a separate pipeline job so we can compare them side by side.

Jobs to create:

- Conditional-1-Test (Method 1)
- Conditional-2-ShellIf (Method 2)
- Conditional-3-GroovyIf (Method 3)
- Conditional-4-Ternary (Method 4)
- Conditional-5-When (Method 5)
- Conditional-6-Scripted (Method 6)

Each job is created the same way:

1. Click **New Item** → Enter name → Select **Pipeline** → **OK**
2. Paste the script → **Save**
3. Click **Build No**

Method 1: sh with test Command (Shell-Level)

```
pipeline {
  agent any
  stages {
    stage('build') {
      steps {
        sh """
          test -z \$SKIP_BUILD && echo 'starting to build ...'
        """
      }
    }
  }
}
```

Why it works:

- test -z $SKIP_BUILD returns true if SKIP_BUILD is empty or unset
- && echo '...' runs the echo only if the test succeeds
- \$SKIP_BUILD escapes the $ so Jenkins passes the variable name to the shell rather than substituting it in Groovy

Method 2: sh with Shell if-else

```
pipeline {
  agent any
  stages {
    stage('build') {
      steps {
        script {
          sh """
            if [ -z \${SKIP_BUILD} ]
            then
              echo "starting build ..."
            else
              echo "skipped build ..."
            fi
          """
        }
      }
    }
  }
}
```

Why it works:

- Shell if [-z ...] tests whether a variable is empty
- then / else / fi are shell keywords for branching
- The script { } wrapper is required in Declarative pipelines for the sh block with variable interpolation

To test the else-branch: Add an environment variable SKIP_BUILD=true in the job configuration (General → This project is parameterized or Environment variables) and rerun. The output will show skipped build ....

Method 3: Groovy if-else in script { }

```
pipeline {
   agent any
   stages {
      stage('build') {
         steps {
            script {
               def skipBuild = env.SKIP_BUILD
               if (skipBuild == null || skipBuild.isEmpty()) {
                  echo 'starting build ...'
               } else {
                  echo 'skipping build ...'
               }
            }
         }
      }
   }
}
```

Why it works:

- def skipBuild = env.SKIP_BUILD reads the environment variable
- if (skipBuild == null || skipBuild.isEmpty()) checks for null or empty
- echo prints the appropriate message
- The script { } wrapper is required in Declarative pipelines

Method 4: Groovy Ternary Operator

```
pipeline {
   agent any
   stages {
      stage('build') {
         steps {
            script {
               def skipBuild = env.SKIP_BUILD
               (skipBuild == null || skipBuild.isEmpty())
                  ? println('starting the build ...')
                  : println('skipping the build ...')
            }
         }
      }
   }
}
```

Method 5: when Directive (Skip Entire Stage)

```
pipeline {
    agent any
    stages {
        stage('build') {
            when {
                expression {
                    def skipBuild = env.SKIP_BUILD
                    return skipBuild == null || skipBuild.isEmpty()
                }
            }
            steps {
                script {
                    echo 'starting build ...'
                }
            }
        }
        stage('report') {
            steps {
                echo 'report stage always runs'
            }
        }
    }
}
```

Why it works:

- when { expression { ... } } evaluates the expression
- If it returns true, the stage runs; if false, the stage is skipped
- Skipped stages appear gray in Stage View with a "skipped" message
- Subsequent stages run normally — when doesn't stop the pipeline

Method 6: Scripted Pipeline with Plain Groovy if-else

```
node {
    stage('build') {
        def skipBuild = env.SKIP_BUILD
        if (skipBuild == null || skipBuild.isEmpty()) {
            echo 'starting build ...'
        } else {
            echo 'skipping build ...'
        }
    }
    stage('report') {
        echo 'report stage always runs'
    }
}
```

Why it works:

- Scripted pipelines use node { } instead of pipeline { }
- Groovy code goes directly inside stage — **no** script { } **wrapper needed**
- This is the key difference: declarative requires script { }, scripted does not

Implementation:

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on jenkins-agent-01 in /home/jenkins/workspace/Conditional-1-Tes
[Pipeline] {
[Pipeline] stage
[Pipeline] { (build)
[Pipeline] sh
+ test -z
+ echo 'starting to build ...'
starting to build ...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
Started by user admin
[Pipeline] Start of Pipeline (hide)
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Conditional-3-GroovyIf
[Pipeline] {
[Pipeline] stage
[Pipeline] { (build)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
starting build ...
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Conditional-3-GroovyIf
[Pipeline] {
[Pipeline] stage
[Pipeline] { (build)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
starting build ...
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (report)
[Pipeline] echo
report stage always runs
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Conclusion

In this article, we learned about using conditional constructs in the Jenkins pipeline. Further, we explored declarative and scripted ways of writing the Jenkins pipeline using if–else blocks, the sh step, and the when directive.

Task 20: Skip a Stage in a Jenkins Pipeline

1. Objective

To conditionally bypass (skip) one or more stages in a Jenkins pipeline using three methods: the when directive (automatic), the input step (manual/user-driven), and a custom reusable function with a closure (DRY pattern).

2. Purpose

Not every stage needs to run on every build. Real-world pipelines benefit from skipping stages for:

- **Speed** — skip long-running tests during rapid development iterations
- **Flexibility** — skip deployment for feature branches or local testing
- **Condition-based logic** — skip deploy when the branch is not main
- **Manual control** — let a human approve or skip a critical stage at runtime
- **Resource conservation** — avoid expensive stages when preconditions aren't met
- **DRY principle** — reuse the same skip logic across multiple stages instead of copy-pasting

3. Concepts Reference

3.1 The Three Methods

Method	Trigger	Best For	Human Needed?
when directive	Automatic (based on condition)	Most cases	❌ No
input step	Manual (user clicks)	Interactive pipelines	✅ Yes
Custom function	Automatic (reusable logic)	Many stages to skip	❌ No

3.2 The when Directive Sub-Conditions

Condition	Meaning
expression { ... }	Custom Groovy expression
branch 'main'	Current branch is main
environment name: 'X', value: 'Y'	Env var matches
not { ... }	Invert condition
anyOf { ... }	Any sub-condition is true
allOf { ... }	All sub-conditions are true

3.3 The input Step

Pauses the pipeline and waits for user action. Can accept parameters and a timeout.

Attribute	Purpose
message	Prompt shown to the user
ok	Label for the confirm button
parameters	Input parameters collected from the user
timeout	Auto-cancel if no response within the time

4. Step-by-Step Procedures

Step 0: Create the Job

1. Click **New Item** → Name: Skip-Stage-Demo → Select **Pipeline** → **OK**

We will reuse this job and update the script for each method.

Method 1: Skip Stage Using the when Directive (Automatic)

```
pipeline {
  agent any
  parameters {
    booleanParam(
      name: 'skip_test',
      defaultValue: false,
      description: 'Set to true to skip the test stage'
    )
  }
  stages {
    stage('Build') {
      steps {
        sh 'echo "Building application"'
      }
```

```
    }
    stage('Test') {
      when {
        expression { params.skip_test != true }
      }
      steps {
        sh 'echo "Testing application"'
      }
    }
    stage('Deploy') {
      steps {
        sh 'echo "Deploying application"'
      }
    }
  }
}
```

Step 1.2: Save → Click **Build with Parameters** →**Build Now** → Leave skiptest → unchecked→**Build**
Step 1.3: Observe the console output:
text
Building application
Testing application
Deploying application
Finished: SUCCESS
Step 1.4: Rerun — this time **check** skip_test → Click **Build**
Step 1.5: Observe the console output:
text
Building application
[Pipeline] stage
[Pipeline] { (Test)
Stage "Test" skipped due to when conditional
[Pipeline] }
Deploying application
Finished: SUCCESS
Why it works:

- The booleanParam provides a user-toggleable flag
- when { expression { params.skip_test != true } } evaluates to false when the parameter is true
- When the condition is false, Jenkins **skips the entire stage** — no steps run
- The pipeline continues to the next stage

When to use: You want to skip a stage automatically based on a parameter, branch, environment, or any other condition.

Method 2: Skip Stage Using the input Step (Manual)

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh 'echo "Building the application"'
```

```
        }
    }
    stage('Test') {
        steps {
            script {
                def userInput = input(
                    message: 'Do you want to run the test stage?',
                    ok: 'Proceed',
                    parameters: [
                        booleanParam(name: 'run_test', defaultValue: true, description: 'Run the test stage?')
                    ],
                    submitter: 'admin',
                    timeout: time(minutes: 5)
                )

                if (userInput) {
                    sh 'echo "Testing the application"'
                } else {
                    echo "Skipping Test stage as requested."
                }
            }
        }
    }
    stage('Deploy') {
        steps {
            sh 'echo "Deploying the application"'
        }
    }
  }
}
```

Step 2.2: Click **Save** → Click **Build Now**
Step 2.3: Observe — the pipeline **pauses** at the Test stage. A dialog appears in the Jenkins UI:
text
Do you want to run the test stage?
[✓] Run the test stage?
[Proceed] [Abort]
Step 2.4: Click **Proceed** with the checkbox checked → Observe the console output:
text
Building the application
Testing the application
Deploying the application
Finished: SUCCESS

Why it works:

- input pauses the pipeline until a user responds
- The parameters list collects user input (here, a boolean)
- The script { } block inspects the response and branches accordingly
- timeout: time(minutes: 5) auto-aborts the stage if no response within 5 minutes (prevents indefinite hangs)

Note on submitter**:** Restricts who can approve the input (e.g., submitter: 'admin'). Remove it to allow any user.

When to use: Interactive pipelines where a human makes the skip decision at runtime.

Caveat: The pipeline will **hang** until someone responds (or the timeout fires). This blocks an executor.

Method 3: Skip Stages Using a Custom Function (Reusable)

```
pipeline {
    agent any
    parameters {
        booleanParam(name: 'skip_test', defaultValue: false, description: 'Skip the test stage')
        booleanParam(name: 'skip_lint', defaultValue: false, description: 'Skip the lint stage')
    }
    stages {
        stage('Build') {
            steps {
                sh 'echo "Building the application"'
            }
        }
        stage('Lint') {
            steps {
                execute_stage('Lint', params.skip_lint) {
                    sh 'echo "Running linter..."'
                }
            }
        }
        stage('Test') {
            steps {
                execute_stage('Test', params.skip_test) {
                    sh 'echo "Running tests..."'
                }
            }
        }
        stage('Deploy') {
            steps {
                sh 'echo "Deploying the application"'
            }
        }
    }
}

def execute_stage(stage_name, skip, Closure steps) {
    if (skip) {
```

```
        echo "⏭ Skipping ${stage_name} stage"
    } else {
        echo "▶ Running ${stage_name} stage"
        steps.call()
    }
}
```

Why it works:

- execute_stage is a Groovy function that takes three arguments:
  - stage_name — for logging
  - skip — boolean flag
  - steps — a **closure** containing the actual stage steps
- If skip is true, the function prints a skip message and does NOT call steps.call()
- If skip is false, the function calls steps.call() which runs the closure

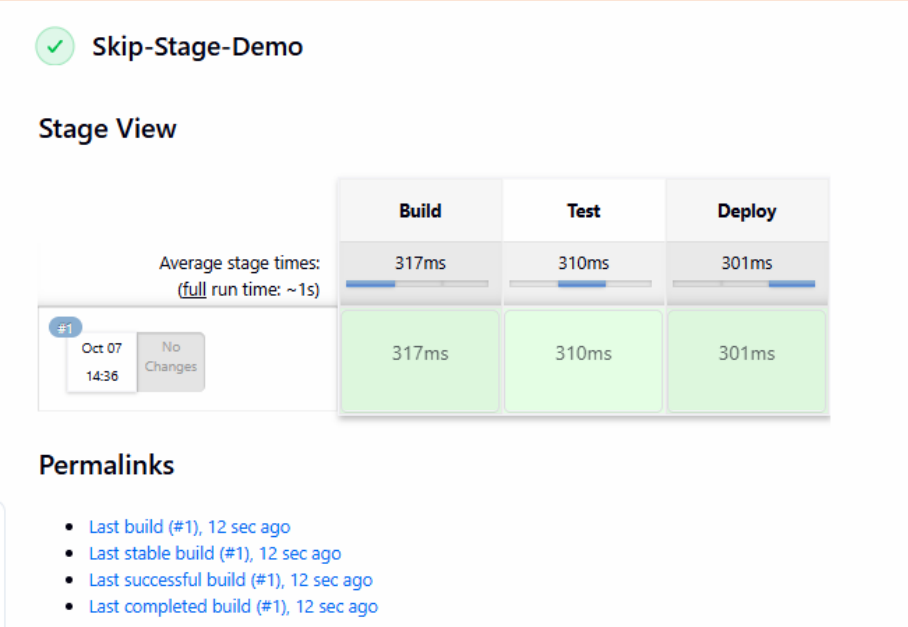
When to use: You have **multiple stages** with the same skip logic. Instead of repeating script { if (!skip) { ... } } in every stage, you write the logic once and pass different closures to it.

Important Groovy note: The function must be defined **outside** the pipeline { } block. Jenkins compiles the entire script, so the function is available to be called from inside stages.

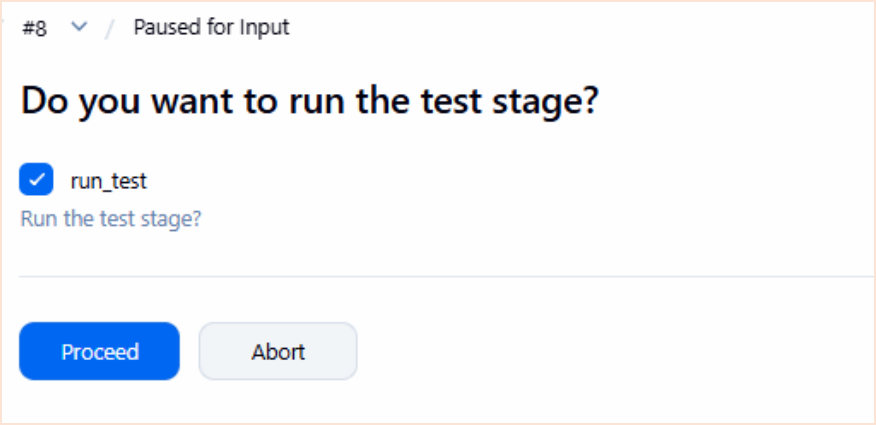
Comparison Table of Skip Methods:

Feature	when Directive	input Step	Custom Function
Automatic	✅	❌	✅
Requires human action	❌	✅	❌
Skips the entire stage	✅	❌ (branches inside)	❌ (branches inside)
Reusable across stages	❌	❌	✅
Blocks executor while waiting	❌	✅ (until timeout)	❌
Shown in Stage View as skipped	✅	❌	❌
Best for	Automatic skips	Interactive approvals	Multiple similar stages

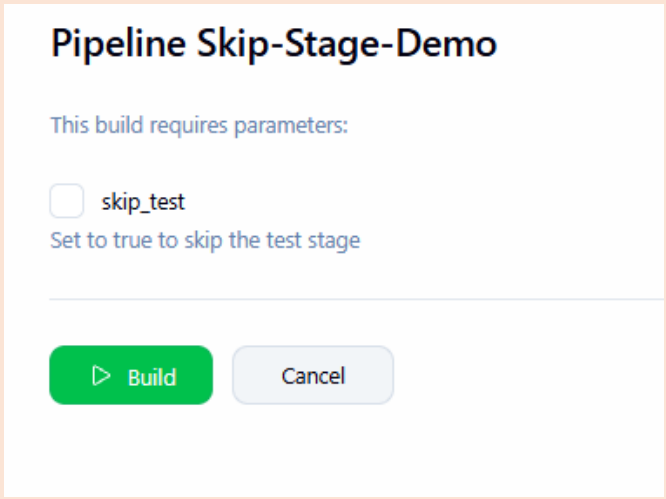
Implemetation:

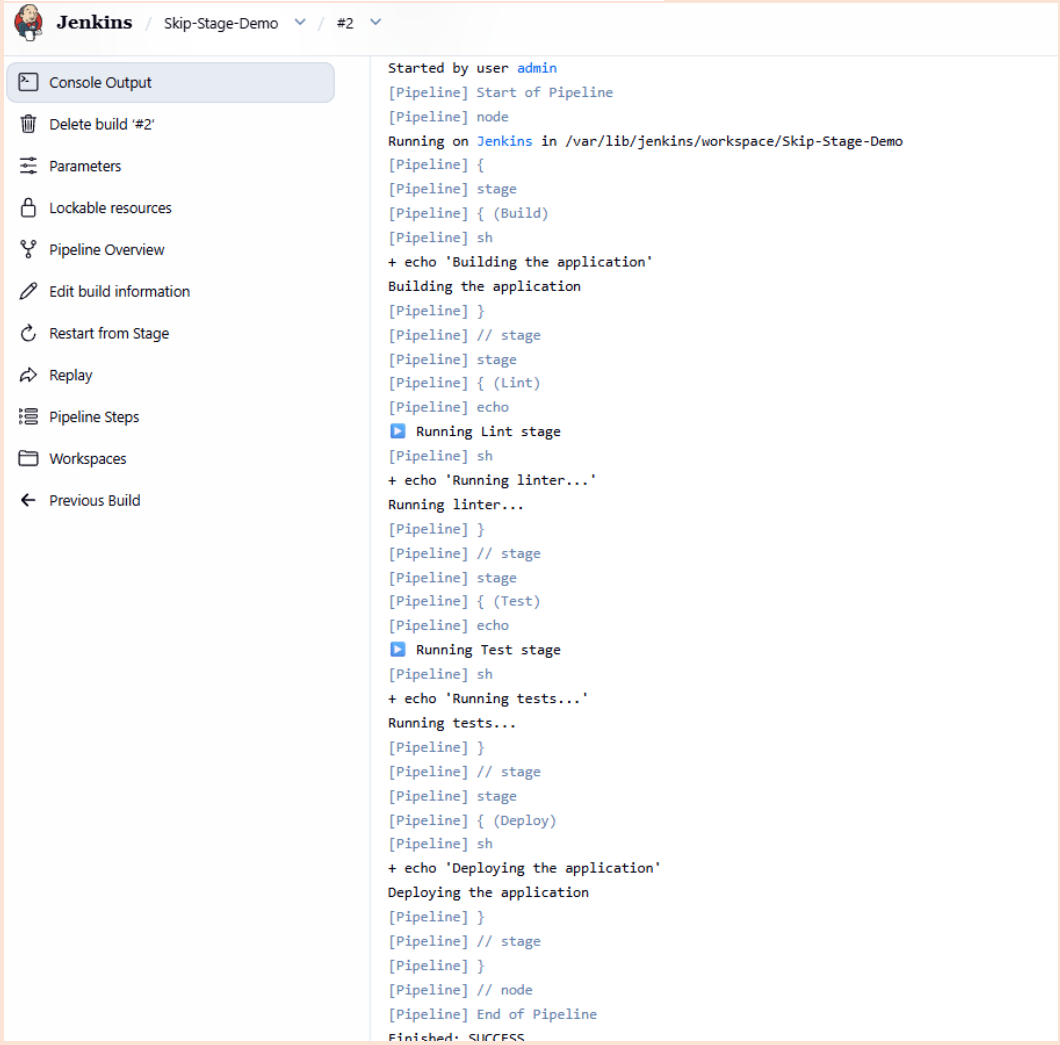



```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Skip-Stage-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] sh
+ echo 'Building application'
Building application
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] sh
+ echo 'Testing application'
Testing application
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] sh
+ echo 'Deploying application'
Deploying application
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```




```
Started by user admin
[Pipeline] Start of Pipeline
[Pipeline] node (hide)
Running on Jenkins in /var/lib/jenkins/workspace/Conditional-3-GroovyIf
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] sh
+ echo 'Building the application'
Building the application
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] script
[Pipeline] {
[Pipeline] input
WARNING: Unknown parameter(s) found for class type 'org.jenkinsci.plugins.workflow.support.steps.input.InputStep': time,unit
Input requested
Approved by admin
[Pipeline] sh
+ echo 'Testing the application'
Testing the application
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] sh
+ echo 'Deploying the application'
Deploying the application
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```





Conclusion

In this article, we discussed three different solutions for skipping stages in a Jenkins-scripted pipeline. First, we explored *when directive* to skip a stage based on a condition. After that, we used the *input step* to prompt the user to skip a stage. Finally, we looked at a custom function to control the flow of the pipeline.

We can also find scripts to skip a stage in the Jenkins pipeline job over on GitHub.